Week 10 Notes

Graphs:

Vertices and Edges, Paths and Reachability,

Colouring, Covers, Matchings,

Representing Graphs, BFS and DFS,
Components, Cycles, Trees, Directed Graphs, Complexity, DAGs and Topological Sorting

Course: Mathematics–1

Notes created by: Vishal (Instructor)

Lecture 1 — Introduction to Graphs

From relations to graphs; directed and undirected graphs; paths, walks, reachability and connectedness

1 Where this fits

For the next few weeks we will be looking at **graphs** — this is the concluding section of the course. We already met graphs, in a sense, in the very first week, when we were talking about **relations**. What we drew there merely as a *visualisation* of a relation will now become an object of study in its own right, and we shall formally analyse what can be done with it.

2 From a relation to a picture

Recall what a relation is. We take two sets — they could even be the same set — form the **Cartesian product** of all pairs, and then select some subset of that product; those pairs are the ones we say are **related**.

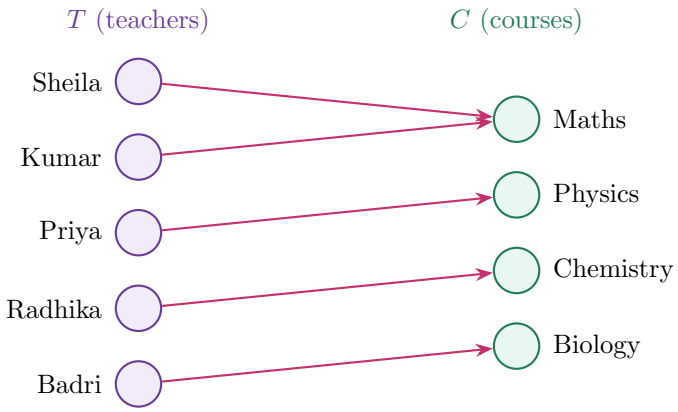
Example 1.1 — teachers and courses

Let T be the set of teachers and C the set of courses offered in the current semester. Then $T \times C$ is the set of all possible pairs whose first element is a teacher and whose second element is a course. A relation

$$R \subseteq T \times C$$

— a kind of **course allocation** relation — describes which teacher is assigned to which course.

We can draw this relation in pictorial form. Create a **node**, or dot, for each element of our sets. Since there are two different types of object here — teachers and courses — we write them in two columns. Whenever a teacher is teaching a course, we connect that teacher to the corresponding course node by an **arrow**.



A small observation

There are 5 teachers and only 4 courses, so clearly **at least one course must be taught by two different teachers**. In this picture, Maths is being taught by both Sheila and Kumar.

3 The definition of a graph

Definition 1.1 — Graph

A **graph** consists of a set of **nodes** (or **vertices**) together with **edges** between them. So a graph has two components:

$$G = (V, E),$$

where V is the set of vertices and E is the set of edges. The edges *connect* the vertices.

On the word *vertex*

Vertices is the plural of *vertex*: one vertex, many vertices. We use **node** and **vertex** interchangeably as names for these elements.

Notice what happens to our earlier picture. There we had two sets, T and C, and a relation which was a subset of $T \times C$. But once we put it into a graph, we **lose — or rather stop caring about — the distinction** between T and C:

$$V = T \cup C,$$

so now there are **nine vertices**, with no real separation between the five that came from the teacher set and the four that came from the course set. The edges are exactly those pairs the original relation contained, namely which teachers are teaching which courses.

In general, then, E is simply a **binary relation on the vertices**: it connects some vertices to some other vertices.

4 Directed graphs

The teacher–course graph has a **direction**: a teacher teaches a course, and there is no corresponding edge from a course back to a teacher. So if (v, v') is an edge, it does not necessarily mean that (v', v) is an edge — here it does not even make sense — although there are other relations where it does.

Definition 1.2 — Directed graph

A graph in which every edge has an **order** — a starting vertex and an ending vertex — is called a **directed graph**. You are supposed to go from the start to the end; you cannot go backwards. Think of it as a **one-way road**.
Pictorially we draw an edge as a line with an arrow. Mathematically it is a **pair**, an element of $V \times V$:

$$e = (v, v') \in V \times V.$$

So the teacher–course graph is **directed**.

5 Undirected graphs

Suppose instead we look at a group of people and try to capture which of them are **friends** of each other. Presumably if Sheila is a friend of Badri, then Badri is also a friend of Sheila — friendship is not a one-directional thing; you cannot be my friend if I am not your friend.

In this case we do not draw an arrow; we just have pairs representing a **symmetric relation**. Recall symmetric relations: if (a, b) is in the relation, then (b, a) is also in the relation.

Definition 1.3 — Undirected graph

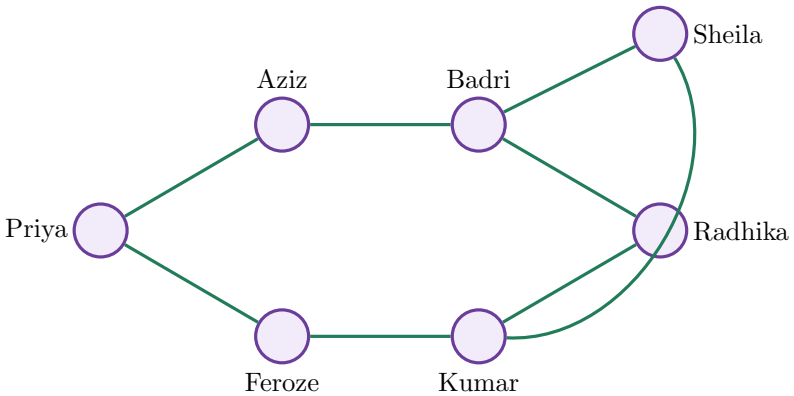
A graph whose edge relation is **symmetric** —

$$(v, v') \in E \implies (v', v) \in E$$

— is called an **undirected graph**, and is drawn with plain lines and *no* arrowheads.

Why treat it separately?

We could always represent a symmetric relation by putting in edges in both directions — nothing stops us from having an edge from Sheila to Badri and another going back. But it is much more convenient to draw a **single edge with no arrows**, indicating that the relation is symmetric. Since this is an important special case, it is usually treated separately.



Type	Edge relation	Example
Directed	not symmetric — the direction is recorded	teacher teaches course; airline routes; parent–child
Undirected	symmetric	friendship

6 Paths

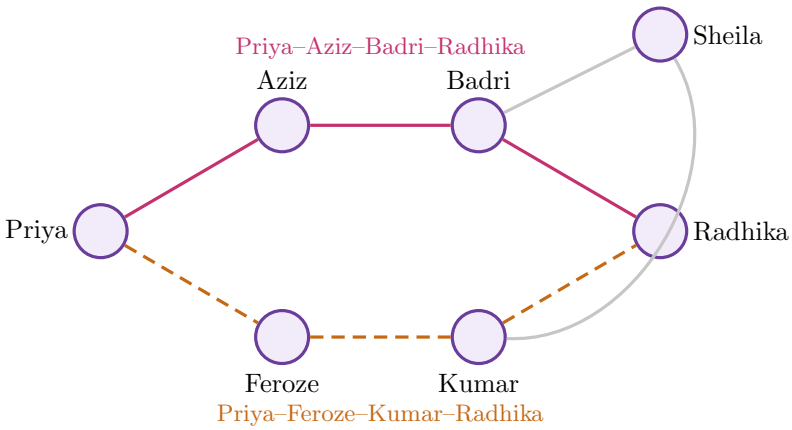
What can we do with such a graph, other than just visualise the relationships between people?

Example 1.2 — Priya needs help

Suppose Priya needs some help, and it turns out that Radhika is in a position to provide it. But as the graph shows, there is **no direct connection** between Priya and Radhika — they are not friends — so Priya may not even be aware that Radhika can be a source of help.

What do we do in real life? When we have a problem we reach out to our friends and ask: *I have a problem, do you know somebody who could help me?* So Priya reaches out to her friends — here Aziz and Feroze — and one of them reaches out to their friends, and so on, until eventually somebody hits upon Radhika.

- One scenario: Priya tells **Aziz**; Aziz tells **Badri**; and Badri says "Oh, I know that Radhika can solve this — let me put Priya in touch with Radhika."
- Another: Priya asks **Feroze**; Feroze propagates the question to **Kumar**; and Kumar, being independently a friend of Radhika, finds out the same thing.



What we have constructed through the friend relation is a **path**: a sequence connecting Priya to Radhika, even though there is no direct relationship between them. This is what the graphical representation buys us — we can look for these **long-distance connections**.

Definition 1.4 — Path

A **path** in a graph is a sequence of vertices

$$v_1,\ v_2,\ v_3,\ \ldots,\ v_k$$

with a starting vertex v_1 and an ending vertex v_k, such that each consecutive pair is an edge:

$$(v_1, v_2) \in E, \quad (v_2, v_3) \in E, \quad \ldots, \quad (v_{k-1}, v_k) \in E.$$

So there are $k-1$ edges connecting these k vertices, and we may travel from v_1 to v_k by following them.

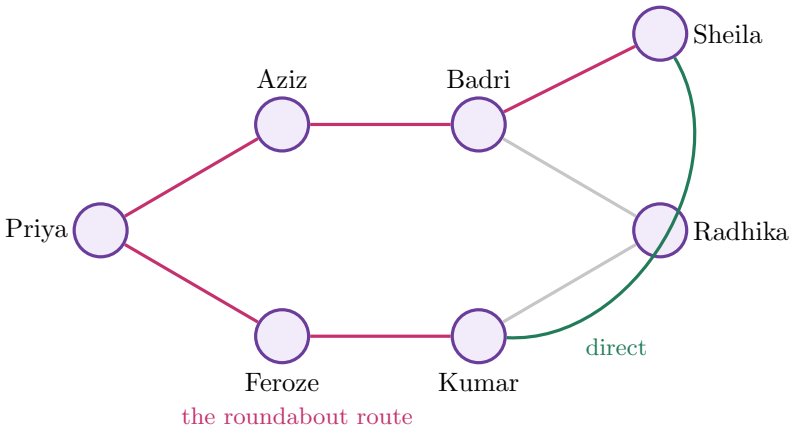
7 Walks versus paths

The definition above says nothing about whether some v_i in the sequence may be the same as a later v_j — in other words, whether we may go to some place in the graph, come back to it, and

then proceed. Of course, one can imagine that this is never necessary.

Example 1.3

Suppose we want to connect **Kumar** to **Sheila**. There is a direct connection. But instead of taking it, we could take a roundabout route: Kumar → Feroze → Priya → Aziz → Badri → Sheila.



Technically, in graph theory a sequence that repeats vertices is *not* a path.

Definition 1.5 — Walk

A sequence which starts at one vertex, ends at another, and **possibly repeats vertices** along the way is called a **walk**. A **path** is the special case in which **no vertex is repeated**.

So a walk is the more general notion; in a path we always assume there are no repeated vertices.

How long can a path be?

Every graph has a finite number of vertices; call it

$$n = |V|.$$

Then a path cannot have more than n vertices — because if it had $n+1$ vertices, some vertex would have to repeat. Consequently, in terms of edges,

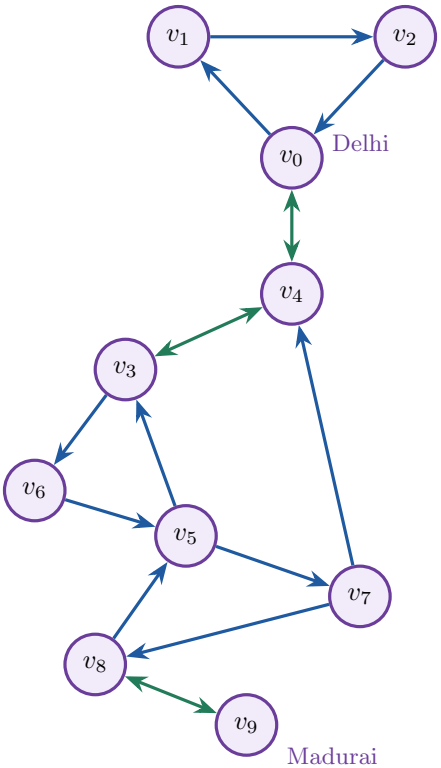
$$\text{a path has at most } \; n-1 \; \text{ edges.}$$

8 Paths in a directed graph: an airline network

The example above was in the friendship graph, but of course we can also talk about paths in **directed** graphs. The teacher–course graph is a bit boring for this purpose, since there is no way to go from a teacher to a course and then anywhere else — you just get stuck. A more typical directed graph is a **transportation network**.

If you have ever looked at an airline map or a railway map, you will have seen graphical representations of the routes served. The picture below is an abstract map of **ten cities** served by some airline. Superimposing it roughly on a map of India, we may take

$$v_0 = \text{Delhi (in the north)}, \qquad v_9 = \text{Madurai (in the south)}.$$



The arrows indicate that **not all flights go between both cities in both directions**. Typically, between large cities — Delhi and Mumbai, say, or Bangalore and Chennai — if an airline flies in one direction it also flies in the reverse direction. But in smaller sectors airlines often operate **triangular routes**: a route serving three cities which does not go back and forth between each pair, but starts in one city, goes to a second, then to a third, and returns to the first.

Reading the arrowheads

Some edges are drawn with **two arrowheads** — for instance between v_4 and v_0, and between v_4 and v_3. This means the graph is directed but we explicitly have an edge in *both* directions. Where there is only **one arrowhead**, the edge is unidirectional: we can go from v_3 to v_6, but we cannot go back from v_6 to v_3.
As a consequence of the triangular route $v_3 \to v_6 \to v_5 \to v_3$: if you want to go from v_3 to v_5, you must sit through a **halt at** v_6.

8.1 Finding a path

Example 1.4

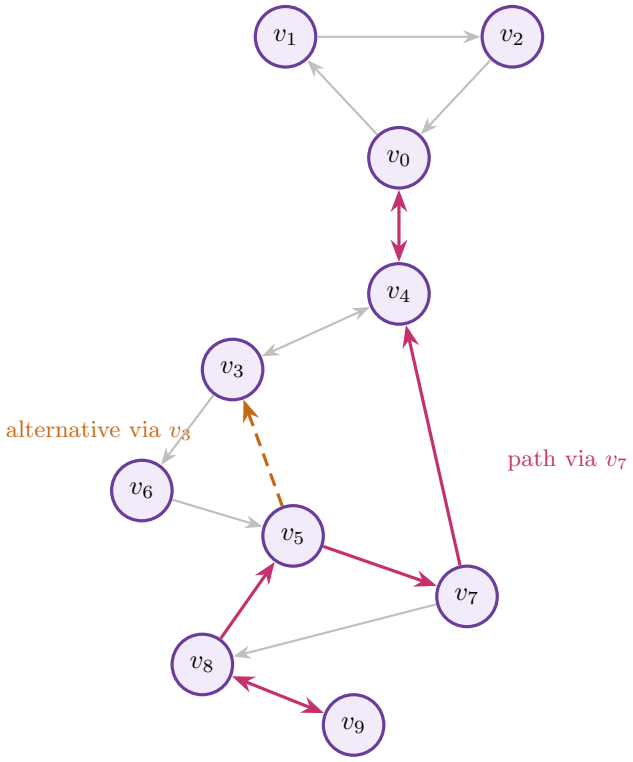
I am in Madurai, that is v_9. Is it possible, on this airline, to travel from v_9 to Delhi, that is v_0?

Looking at the picture, here is one possible path:

$$v_9 \to v_8 \to v_5 \to v_7 \to v_4 \to v_0.$$

But this is obviously not the only way. From v_9 we have no choice — we can only go to v_8 — and from v_8 we can only go back to v_9 (not very useful) or on to v_5. At v_5, however, there is an **option**: we may go to v_3, and v_3 is connected *both ways* to v_4, so we can come back to v_4 that way instead:

$$v_9 \to v_8 \to v_5 \to v_3 \to v_4 \to v_0.$$



So there are **two ways** to get to v_4: either via v_3 or via v_7. Both have roughly the same number of cities in between, so there is not much advantage either way — but there are multiple paths, and we can discover them.

9 Reachability

Definition 1.6 — Reachable

In a graph we say that a vertex v is **reachable** from a vertex u if there is a **path** from u to v.

10 Questions we might ask about a graph

Typical questions

1. **Is a vertex v reachable from u?** This is exactly what we asked about Madurai and Delhi, and about Priya and Radhika — a reachability question for a *specific pair* of vertices.
2. **What is the shortest path?** Given that travel is possible, we may still want the best way to do it — say, the fewest flights.
3. **Starting from u, where can I reach?** This is a more general question: knowing the whole set of reachable vertices answers question 1 as a special case.
4. **Is the graph connected?** Can I go from everywhere to everywhere?

On shortest paths

For now, "shortest" means simply the number of **edges**, or the number of intermediate vertices. Later we shall see that we can associate a **distance** or a **time** with each leg, and then ask for

the shortest path not in terms of the number of hops but in terms of that quantity.
This is exactly what a travel website does: it offers you a direct flight, a one-hop flight, a two-hop flight, and you might prefer the one with fewer stops so as not to waste time waiting on the ground.

11 Connectedness

For an **undirected** graph, connectedness simply means that starting at one vertex we can reach every other vertex. For a **directed** graph it is a little more complicated: we may be able to go from one vertex to every other vertex and yet not be able to *come back*. So we require that **from every vertex we can reach every other vertex**.

11.1 The airline graph is connected

In the original graph you can check that you can go from everywhere to everywhere. There are three **sections** — the triangles $\{v_0, v_1, v_2\}$, $\{v_3, v_6, v_5\}$ and the lower group — and within each you can go around the circuit and reach any place you want. The sections are joined by the **bidirectional** edges at v_4, so from each component you can get to every other component.

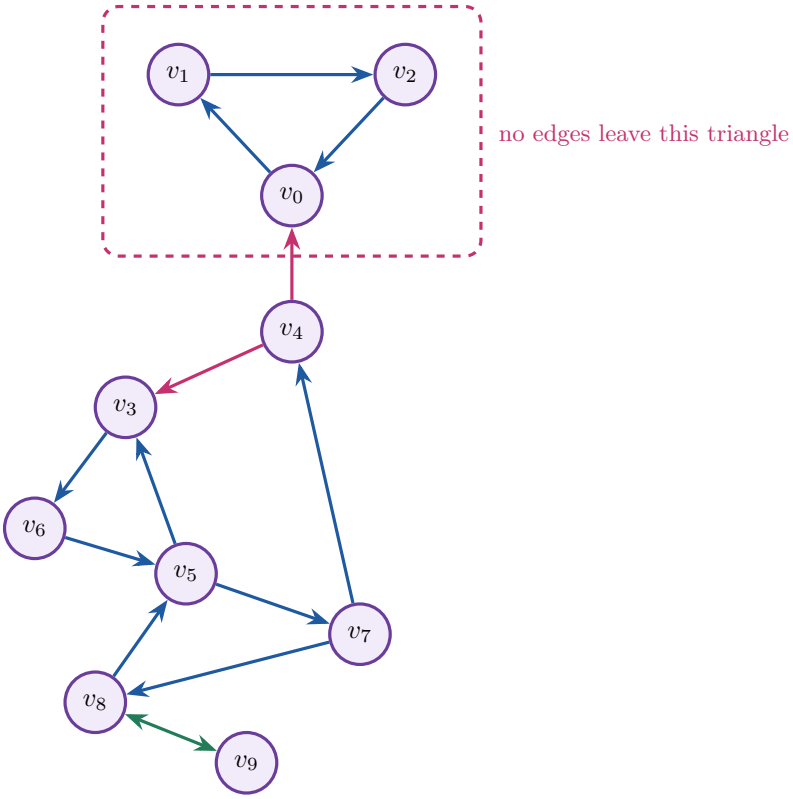
11.2 Breaking one direction: still connected

Now suppose we take the v_4–v_3 connection and make it **one-directional**, so that we can only go from v_4 to v_3 and not back. Earlier we could use v_3 as an escape route out of that triangle; now we cannot. But we can still go down from v_5 to v_7 and proceed from there.

So even after making $v_4 \to v_3$ one-directional, there is **no problem**: the graph is still connected — from every city we can reach every other city.

11.3 Breaking a second direction: no longer connected

However, if we now take another edge at v_4 — say v_4–v_0 — and also make it a single direction $v_4 \to v_0$, then we have an issue. We can go *up* through that edge from everywhere below; but once we are at the top, we **cannot come back down**. There is no way to leave the triangle v_0, v_1, v_2, because no edges leave it.



So the graph has now become **not connected**, in one specific way: the vertices v_0, v_1, v_2 cannot reach the other vertices. Everything else *can* reach them — so notice that this is an **asymmetric** failure; the other vertices can still reach v_0, v_1, v_2.

We are only posing the questions

We are not solving these questions here — we are simply posing them as the typical questions one might want to ask once a graph is presented as a representation of some information. That information may come from a relation, as it did for us, or it may be naturally given, like friendships or airline routes.

12 Summary

A **graph** represents a **relationship between entities**:

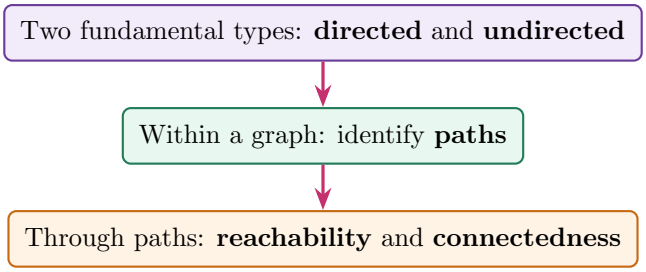
- the entities are represented as **nodes** or **vertices**;
- the relationship is represented by **edges** between them;
- these edges may be **directed** or **undirected**.

Example 1.5 — another directed graph: family relationships

Take a group of people and connect those who are **parent and child**: if A is a parent of B, then A is related to B. Clearly this is **asymmetric** — if A is a parent of B, then there is no way that B can be a parent of A.

You may have seen such a graph pictorially in the form of a **family tree**: people sometimes represent relationships within a family by drawing edges between parents and children, with some way of connecting people who are married, and so on. A family tree is a kind of graph, and it is typically **directed**, because the parent–child relation is asymmetric.

On the other hand, as we saw, a **friend** relation gives an **undirected** graph.



These are the problems of interest to us right now, and we shall explore them more systematically in the lectures to come.

Lecture 2 — What Else Can Graphs Do?

Map colouring and graph colouring; planar graphs; scheduling; vertex cover; independent sets; matchings

1 Recalling the first lecture

In the first lecture we introduced the concept of a **graph**: a graph consists of a set of **vertices** and a set of **edges**, the edges being pairs of vertices, so that the edge relation is a subset of $V \times V$. Our running directed example was a graph of airline routes.

We then said that a **path** in a graph is a sequence of edges leading from one vertex to another, with no vertex repeated in between — each edge must be an extension of the previous one, as in $v_9 \to v_8 \to v_5 \to \cdots \to v_0$. And we talked about **reachability**: given a vertex u, can we reach a vertex v by finding a path?

> So far the only problem we have really looked at is **reachability**, and it is indeed one we shall spend time on. But before getting into how to calculate reachability, it is worth seeing that graphs have far more interesting problems associated with them: having a graph representation of a problem lets you deal with a great many different scenarios.

2 Colouring a map

Let us start with a problem which does not appear to be connected to graphs at all: **how to colour a map**.

When you look at a political map of the world, or of a country like India, each political unit has a different colour — but *not* all of them have different colours. Some colours are repeated in different places.

The rule of map colouring

When we colour a state (or country) on a map, it must have a **different colour from every state which shares a border** with it — so that there is no confusion at the border, one side being coloured one way and the other side the other way.

The natural question — and at the moment it looks like an idle mathematical question — is: **how many colours do we need?**

Clearly, if there are a certain number of states we could just use a different colour for every state. That gives an **upper bound**: with 27 states, 27 colours certainly suffice, since then no two states share a colour at all, let alone two bordering ones. But perhaps we do not need 27. Can we do better?

3 Turning the map into a graph

This is where a graph comes in. How do we build one?

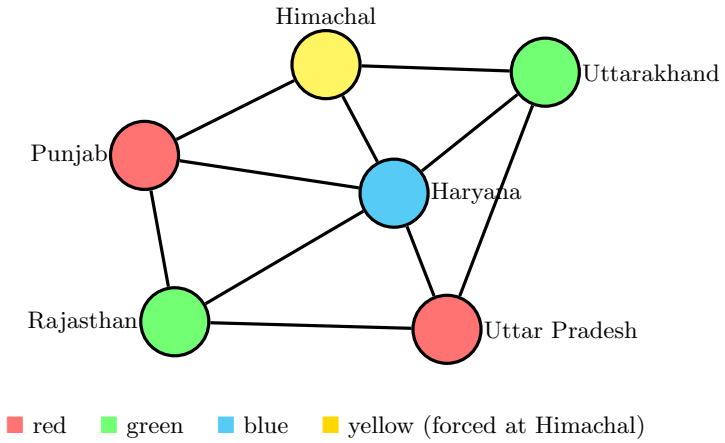
- **Vertices:** the states (or, for a world map, the countries). For every state there is one black dot.
- **Edges:** the edge relation represents **sharing a border** — being neighbours, and therefore having to be coloured differently. So we connect every state to each of its neighbours.

Our goal — associating a colour with every state on the map — is now exactly the same as associating a colour with every dot in this graph.

3.1 Colouring it step by step

Start by assigning a colour, say **red**, to Uttar Pradesh. The rule now tells us that every neighbouring state — anything connected to Uttar Pradesh in the graph — must have a different colour.

- Choose **green** for Uttarakhand and **blue** for Haryana.
- Proceeding to their neighbours: at Rajasthan we can **reuse green**, because green was used for Uttarakhand, and Uttarakhand and Rajasthan do not share a border — so there is no confusion. A colour may be reused as long as it is not being used for any neighbouring state.
- Similarly we can **reuse red** for Punjab, since Punjab is not connected to Uttar Pradesh.
- But when we come to **Himachal** we have a problem: it is surrounded by three neighbours which have already been given three different colours — Punjab is red, Haryana is blue, Uttarakhand is green. So we must choose a **fourth colour**, say yellow.



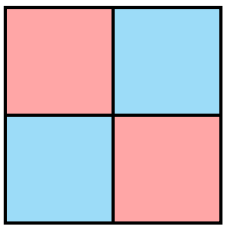
We keep proceeding in this way: whenever we need a new colour we use one, and whenever we can reuse a colour we reuse it. In this way we arrive at a **less expensive** colouring, in terms of the number of colours used.

4 Why the graph is better than the map

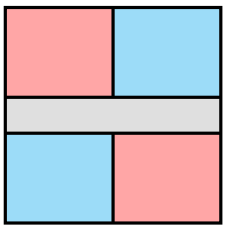
Notice that once we have constructed this graph, describing which states share a border, we do **not really need the map any more**. We may as well colour the graph directly, using the rule: if I colour a node with one colour, then any node joined to it by an edge must get a different colour.

Ambiguity at the borders

Working with the physical map, we have to stare at the borders, and how well we can tell what is happening depends on how well the map is drawn. Sometimes borders **meet at a corner**, and technically the colouring rule does *not* apply across a corner: if four states meet at a point — which actually happens in the United States — then opposite states may share a colour. Using red, red, blue, blue is perfectly legal.
If two states touch only at a **single point**, they are not considered to share a border. But that may depend on how the border is drawn: what looks like a single point may, when blown up, turn out to be a **segment** of common border — and then blue, blue is no longer allowed.



meeting at a point:
red, red, blue, blue is legal



blown up:
now blue, blue is *not* allowed

Once the information has been transferred to the graph, this confusion disappears. In fact we can then take the graph and even **distort** it: the original drawing may follow the geometry of the country faithfully, but we are free to take nodes that are bunched up and move them far apart so as to draw the colouring more clearly.

This is the real advantage of moving to the graph: the graph **abstractly represents the relevant information**, so we can work with whatever format of that information is convenient, without worrying about the original format in which it came. Here the geography and the geometry of the actual state boundaries are no longer important — we only need the **connectivity**, that is, which states are neighbours of which.

5 The graph colouring problem

Abstractly, we have transformed our map colouring problem into a **graph colouring problem**.

Definition 2.1 — Graph colouring

We are given a graph $G = (V, E)$ as before, and separately a set of **colours** C. A **colouring** is a function

$$c : V \longrightarrow C$$

assigning to every vertex a colour from C, subject to the rule that the endpoints of an edge must have different colours:

$$\boxed{(u, v) \in E \quad \Longrightarrow \quad c(u) \neq c(v).}$$

And the question we were asking is: if we do **not** fix the set of colours in advance — as in our example, where we started with red, were forced to choose green, then forced to choose blue, then forced to choose a fourth colour yellow — and instead add colours only as we need them, then **how many colours will we need?** That is, what is the **minimum number of colours** for this particular graph?

6 The Four Colour Theorem and planar graphs

It turns out that this problem has been very well studied for the graphs which come from maps.

The Four Colour Theorem

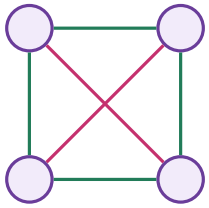
For graphs derived from geographical maps in the way we have described, **four colours suffice**.

This is a very well known theorem, and it is **very hard to prove**. Technically, the graphs it applies to are what are called **planar graphs**.

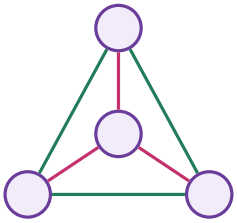
Definition 2.2 — Planar graph

A graph is **planar** if it can be drawn in the plane so that the edges do **not cross**.

Note that a crossing in one particular drawing is not necessarily a problem: an edge which crosses another can often be **redrawn around** the outside, so the graph is still planar. But if we now bring in a further vertex outside and try to connect it across, we may run into trouble — some graphs simply **cannot** be drawn on a sheet of paper without edges crossing, and those are called **non-planar** graphs.



in this drawing
two edges cross ...



... but the same graph redrawn
has no crossings: still **planar**

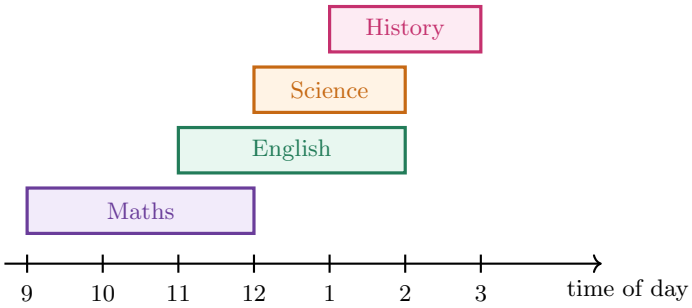
Now, when a map is laid out and we start connecting states, a state obviously cannot share a border with something far away — so a **map always gives a planar graph**. Not all graphs are planar, and the graph colouring question is asked in the general case.

7 Why should we care? Classroom scheduling

Map colouring on its own may look like a specialised application. Why should we care in general how many colours a graph needs? Because graph colouring turns out to be useful in a number of very different situations.

Example 2.1 — how many classrooms?

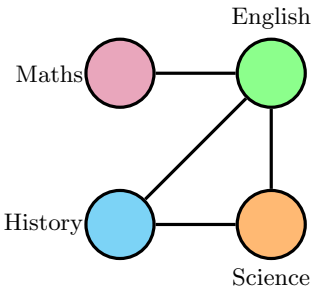
Suppose we are running a school and need to determine how many **classrooms** we need in order to run our classes. This depends on the **timetable**: across the day there are lecture slots, and different courses occupy different slots.



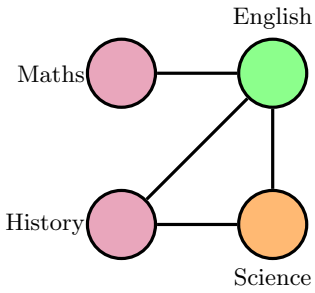
Clearly, if Maths runs from 9 to 12 and English starts at 11, then English must be in a **different classroom** from Maths. Similarly, if History runs from 1 to 3 while Science has already started at 12 and goes on till 2, then History and Science cannot be in the same classroom. So whenever we have **overlapping slots**, the corresponding classes need different classrooms.

The question: what is the **minimum number of classrooms** needed to run all these classes with no scheduling conflicts? As before, we *could* simply give every course its own room — a separate English classroom, a separate Maths classroom, and so on — and the problem is solved. But we want to **optimise**; we do not necessarily want to allocate a separate classroom for every course.

- **Vertices:** the courses.
- **Edges:** an **overlap** — these two courses share a time slot, so they cannot both be scheduled in the same classroom.
- **Colours:** the classrooms.



four colours: one classroom each



Maths and History do not overlap
⇒ same colour ⇒ **three** classrooms

Starting from four different colours on the four nodes — every class in its own room — we observe that **Maths and History do not overlap**. So by graph colouring the same colour can be assigned to both nodes, and one classroom is saved. In this way many **scheduling** problems can be converted into graph colouring problems.

8 Vertex cover: security cameras

Example 2.2 — cameras in a hotel

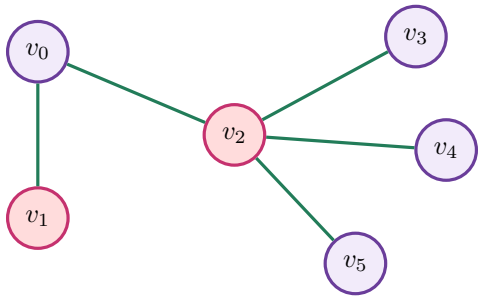
A hotel wants to install **security cameras** in the corridors of a floor. The corridors are straight, but there may be a maze of them meeting at various corners. Assume that a camera placed at an intersection can monitor **every corridor that meets at that intersection**. What is the minimum number of cameras needed to monitor all the corridors on the floor?

Once again we go to graph theory and represent the floor plan as a graph. The points of interest are the **intersections** — it is clearly to our advantage to put a camera at an intersection, since from there it monitors several corridors at once.

- **Vertices:** the intersections.
- **Edges:** the segments of corridor running from one intersection to another.

Definition 2.3 — Vertex cover

A **vertex cover** is a subset of the vertices such that those chosen vertices **cover all the edges** of the graph — that is, every edge has at least one of its endpoints in the subset.



vertex cover = $\{v_1, v_2\}$

Looking at this graph on six vertices $v_0, \ldots, v_5$ — our hotel corridors — suppose we put a camera at v_2. That covers **four** corridor segments, but it does **not** cover the segment from v_0 to v_1. So we must put one more camera; we may choose either v_0 or v_1, and let us say we choose v_1. Now every corridor is covered, and

$$\{v_1, v_2\}$$

is a vertex cover.

Another use of the same problem

This is a problem in graph theory which you can solve **independently of the source**. A similar situation: if you want to locate **ambulances** at intersections so that they can reach all localities quickly, where should you place them on the city map so that every locality is served within a reasonable time?

9 Independent set: the dance programme

Example 2.3 — the school of dance

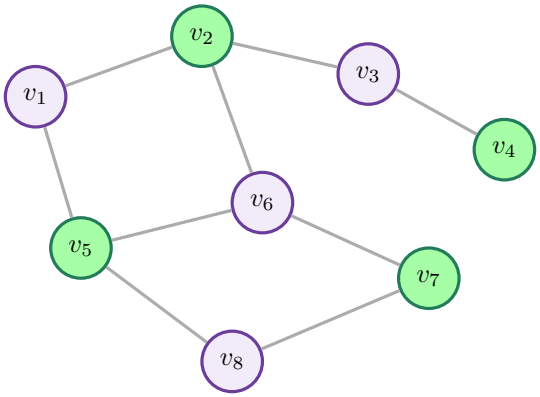
A famous school of dance is going to put up a show consisting of a number of **group dances**. Over time the school has rehearsed many such dances, and **some dancers take part in more than one**. Because of costume changes and other constraints, we would like each dancer to participate in **at most one** dance — we do not want a dancer to perform, go back, change costume and return for another dance. Given which dancers are required for each dance, can we choose a large set of dances for the programme so that no dancer has to dance twice?

- **Vertices:** the dances.
- **Edges:** an **overlap** in the dance group — if two dances share a common dancer, we cannot put both into the programme.

Definition 2.4 — Independent set

An **independent set** is a set of vertices such that there is **no edge between any two vertices in the set**.

So we want to pick a set of dances such that no two of them share a dancer.



independent set $= \{v_2, v_5, v_7, v_4\}$

Here is how we build it on this graph with eight nodes. Suppose we pick v_2 for our independent set. Then nothing connected to it may be used: v_6 is out, v_3 is out, v_1 is out. What next? We may choose v_5 — which rules out v_6 and v_1 (already gone) but also rules out v_8. We can then take v_7, which brings no further constraints, and finally v_4.

So in this scenario we can choose **four** of the vertices with no edge between any of them:

$$\{v_2, v_5, v_7, v_4\}.$$

Of course there is another independent set here as well — the remaining (grey) vertices $\{v_1, v_3, v_6, v_8\}$ also form one.

10 Matching: allocating project groups

Example 2.4 — class projects

A teacher assigns class projects, allowing a project to be done by a single student or by **two** students — but with the constraint that if two people work together they must be **friends**, since otherwise the project would not get done. Given the friendship graph, we want a good allocation of groups.

Note the constraint carefully. If a, b and c are all friends of each other and we pair a with b, then b cannot also partner c; c must find a different partner, and can partner neither a nor b.

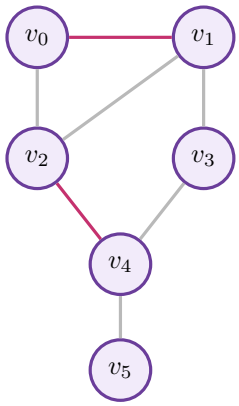
Definition 2.5 — Matching

A **matching** is a subset of edges which is **mutually disjoint**: if we pick one edge and another edge, they do not share any vertex.

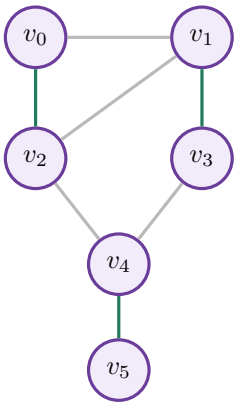
Definition 2.6 — Maximal and perfect matchings

A **maximal matching** is one which **cannot be extended** by adding any more pairs without violating the condition.
A **perfect matching** is a matching which **touches every vertex** of the graph — so every

vertex is part of some pair.



a **maximal** matching: only two pairs



a **perfect** matching: three pairs

Look at the left-hand picture. If we pick the edge v_0v_1, we can no longer pick any edge touching v_0 or v_1. We then pick v_2v_4 — and this knocks out the remaining edges at v_3 and v_4. At this point every edge has been either ruled out or included, and we cannot proceed. So this is a **maximal matching**, and we are stuck making only **two** pairs among these six students.

Ideally, with six students we should be able to make **three** pairs — and that is what a **perfect matching** is, shown on the right: six vertices, three edges, mutually disjoint, with every vertex part of a pair.

11 Why all this matters

Graphs are **not just about connectivity and reachability**. Those are certainly very important problems, but they are not the beginning and end of graphs: there are very many interesting problems one can frame once a problem is put into graph-theoretic form.

Problem	What is asked	Example seen
Graph colouring	fewest colours, adjacent vertices differing	map colouring; classroom scheduling
Vertex cover	fewest vertices covering all edges	security cameras; ambulance placement
Independent set	largest set with no edge inside	choosing dances so no dancer repeats
Matching	disjoint edges; maximal, perfect	allocating project pairs in a class

Although we shall not necessarily look at all of these problems in detail in this course, it is important to understand **why there is so much emphasis on graphs** — and on procedures and algorithms involving graphs. The underlying representation of a graph is a very **rich** representation, and by solving problems in this abstract world of graphs one can solve a number of concrete problems **in one shot**, without having to solve each of them individually.

Lecture 3 — Representing a Graph

Adjacency matrices and adjacency lists; degree; computing reachability by marking

1 Why we need a representation

We have been talking about graphs and problems on graphs. We started with **reachability**, and then looked at rather complicated problems like graph colouring and vertex cover. Let us now come back to reachability and connectivity, and ask a more fundamental question: **how do we actually work with these graphs in a mathematical setting?**

Recall that a graph consists of two sets — or rather a set and a relation: a set of **vertices** and an **edge relation**. A **path** is a sequence of connected edges, and v is **reachable** from u if there is a path from u to v.

As humans, when we see a graph drawn on the page, what we do is **stare at the picture** and extract the information visually. By trial and error — by exploration — we can see that there is a path from v_9 to v_0.

> The problem is that we want to operate on this **mathematically**. We do not want to depend on the picture, because there is no way to formally represent a picture, nor to say how one operates on it in terms of a **procedure** that can be executed. So: how do we represent this picture in a way that lets us *compute* reachability?

2 Conventions and notation

Naming the vertices

Let the set of vertices consist of n vertices. It is the normal convention in graph theory to use

$$n = \text{number of vertices}, \qquad m = \text{number of edges}.$$

For simplicity we **number** the vertices. One could call them 1 to n, but in computing it is very common to start numbering at 0, so we call them

$$0,\ 1,\ 2,\ \ldots,\ n-1.$$

Of course, when we have real vertices — names of cities such as Delhi and Madurai — we keep a **table** mapping the actual names to the numbers:

$$\text{Delhi} = v_0, \qquad \text{Madurai} = v_9, \qquad \ldots$$

In this representation an **edge is a pair of numbers** (i, j), where i and j both lie in the range 0 to $n-1$.

No self-loops

We usually assume that we do **not** have an edge from a vertex to itself — a so-called **self-loop**. So whenever we write an edge (i, j) we assume

$$i \neq j.$$

3 The adjacency matrix

Definition 3.1 — Adjacency matrix

The **adjacency matrix** A of a graph on n vertices has rows and columns numbered 0 to $n-1$, with

$$A[i][j] = \begin{cases} 1, & \text{if there is an edge from } i \text{ to } j, \\ 0, & \text{otherwise.} \end{cases}$$

The default is 0, and we put a 1 wherever there is an edge.

For our airline graph there are 10 vertices numbered 0 to 9, so the matrix has 10 rows and 10 columns. The row and column headers are shown in red; the entry at row i, column j records the edge from i to j.

Adjacency matrix — the *directed* airline graph

	0	1	2	3	4	5	6	7	8	9
0	0	1	0	0	1	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	0	1	0	0	0
4	1	0	0	1	0	0	0	0	0	0
5	0	0	0	1	0	0	0	1	0	0
6	0	0	0	0	0	1	0	0	0	0
7	0	0	0	0	1	0	0	0	1	0
8	0	0	0	0	0	1	0	0	0	1
9	0	0	0	0	0	0	0	0	1	0

Take, for example, the edge from 8 to 5. This says that row 8, column 5 must be a 1 — and indeed looking at row 8 and column 5 we find a 1 at that position. That is how every entry is filled, and you can check that this matrix represents all the edges of the graph.

The bidirectional edges are really two edges

Looking at the graph, 0 is connected to 1 and to 4, and indeed row 0 has 1s in columns 1 and 4. The **bidirectional** edge between 0 and 4 is actually *two* edges — this is a directed graph, and drawing an arrow at both ends is only a shortcut. So there is an edge from 0 to 4 *and* a separate edge from 4 to 0, giving 1s at both $A[0][4]$ and $A[4][0]$.
By contrast there is an edge from 0 to 1 but **no** corresponding edge from 1 to 0, so $A[1][0] = 0$.

3.1 The undirected version

Now suppose that all the routes are in fact bidirectional — whenever the airline flies from one city to another, it also flies back. Then we need not record the directions any more, and it is better to call the graph **undirected** and draw each edge as a plain line.

Adjacency matrix — the *undirected* airline graph

	0	1	2	3	4	5	6	7	8	9
0	0	1	1	0	1	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
2	1	1	0	0	0	0	0	0	0	0
3	0	0	0	0	1	1	1	0	0	0
4	1	0	0	1	0	0	0	1	0	0
5	0	0	0	1	0	0	1	1	1	0
6	0	0	0	1	0	1	0	0	0	0
7	0	0	0	0	1	1	0	0	1	0
8	0	0	0	0	0	1	0	1	0	1
9	0	0	0	0	0	0	0	0	1	0

Here, every time there is an edge from i to j there must also be an edge from j to i, because the edge relation is symmetric. So if we run down the **main diagonal** and look at any entry, the symmetric entry on the other side must be the same:

$$A[i][j] = A[j][i].$$

If there is an edge from 6 to 3 there must be one from 3 to 6; if there is an edge from 2 to 1 there must be one from 1 to 2; and so on.

An adjacency matrix is very simple: create a row and a column for every node, and at the intersection of a row and a column put a 0 if there is no edge and a 1 if there is. That is all there is to it.

4 Computing with the matrix

The whole purpose of doing this is that we can now **throw away the picture** and look only at the matrix.

4.1 Finding neighbours

In the **undirected** graph, if we want to know the neighbours of i — the vertices to which i is joined by an edge — we go to the **row** for i. Looking for the neighbours of 6, we go to row 6 and find a 1 in column 3 and a 1 in column 5. So

$$\text{neighbours of } 6 = \{3, 5\},$$

which, if we look back at the picture, is indeed the case. Without looking at the picture at all we can mechanically analyse the matrix and get the same information — and this is exactly what we need, since there is no way to design a computational procedure that looks at a picture and makes decisions from it.

Directed graphs: rows out, columns in

For a **directed** graph the notion of a neighbour is a little more complex, because there are **outgoing** and **incoming** edges.

rows = outgoing edges, **columns** = incoming edges.

Taking vertex 6: the *row* for 6 has an entry pointing to 5, because there is an edge from 6 to 5. The edge from 3 is coming *in*, so it is not an outgoing edge — to see it we look at the *column*

for 6, which has an entry in row 3.
In an undirected graph these are symmetric, so both carry the same information: conventionally we look at the row, but the column would do just as well. In a directed graph they are different.

4.2 Degree

Definition 3.2 — Degree

The number of neighbours of a vertex is called its **degree**: the number of edges incident on that vertex.

For example, the degree of 6 is 2 — whether we look at the row for 6 (two outgoing edges, to 3 and 5) or at the column for 6 (two incoming edges, from 3 and 5). In an undirected graph there is a uniform notion of degree; it does not matter whether we count edges coming in or going out.

Notice that each edge must go to a *different* vertex, and apart from the vertex we start at there are $n-1$ other vertices. Hence

$$0 \quad \leq \quad \text{degree} \quad \leq \quad n \;-\; 1$$

- **Degree** 0**:** the vertex is not connected to anybody — a friendless soul, or a city not served by this airline at all.
- **Degree** $n-1$**:** everybody in the class is my friend; or I have a direct flight to every other city on the network.

In-degree and out-degree

For a **directed** graph this notion splits, because there are incoming and outgoing edges, and we speak of the **in-degree** and the **out-degree**. The degree of 6 in the undirected setting was 2; but in the directed graph one edge points *out* to 5 and one points *in* from 3, so

$$\text{in-degree}(6) = 1, \qquad \text{out-degree}(6) = 1.$$

5 Computing reachability by marking

Our goal was to *compute*. So: how can we use the adjacency matrix to check whether Delhi — vertex 0 — is reachable from Madurai — vertex 9?

We do the natural thing: start at 9 and see where all we can go. By **marking** a vertex we simply mean recolouring its row header, from red to blue, to record that it has been reached.

Step	Explore	Row gives neighbours	Newly marked
0	— (source)	—	9
1	9	{8}	8
2	8	{5, 7, 9}	5, 7 (9 already marked)
3	5	{3, 6, 7, 8}	3, 6 (7, 8 already marked)
4	3	{4, 5, 6}	4 (5, 6 already marked)
5	4	{0, 3, 7}	**0 — target reached, stop**

Reading the steps in words:

- First we mark that from 9 we can reach 9 itself. Then we look at the neighbours of 9: there is only one, namely 8. Since 9 is reachable and we can fly from 9 to 8, vertex 8 is also reachable, so we mark it.
- Now we must start from 8 and see where all we can go — systematically marking all the neighbours of marked vertices. Vertex 8 has three neighbours, 5, 7 and 9. Note that we **do not have to refer to the picture**: we only look at the row for 8, and that row tells us what its neighbours are. Marking 9 again has no effect, so the new ones are 5 and 7.
- Pick one of the newly discovered vertices, say 5. Its neighbours are 3, 6, 7 and 8, of which 7 and 8 are already known — so 3 and 6 get marked.
- Going to the smallest unexplored vertex, 3: its neighbours are 4, 5 and 6, of which 5 and 6 were already marked, so only 4 is new.
- The smallest unexplored vertex is now 4, whose neighbours are 0, 3 and 7. So 0 gets marked — and since 0 was our target, we can **stop**.

By systematically marking everything reachable in one step, then two steps, and so on, we have found a way of reaching 0 from 9. We may or may not be able to reach 1 and 2 as well; we do not need to know for this particular calculation. Had we wanted *everything* reachable from 9, we would have continued and seen whether 1 and 2 get marked.

And if the target is not reachable? Then after marking everything that can be marked, we simply find that 0 is **still unmarked**. If at the end of the process we cannot mark anything more and some vertex remains unmarked, then that vertex is **not reachable** from where we started.

The procedure, abstractly

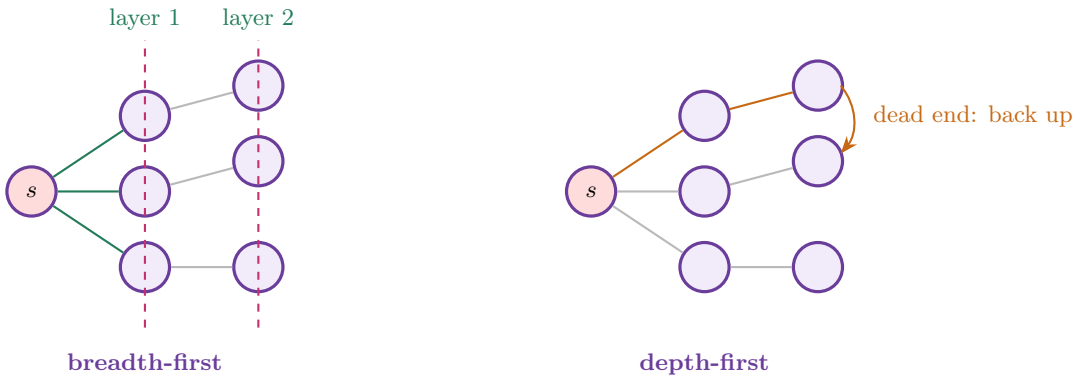
1. Mark the **starting** (source) vertex as reachable.
2. Systematically mark the **neighbours of marked vertices**.
3. Stop when the **target** becomes marked — or when nothing more can be marked.

The important point is that we did all of this **using the matrix**: we never went back to the picture and tried to follow edges by eye. We scanned rows and recoloured row headers, and in that process explicitly computed whether 0 can be reached from 9.

6 Two strategies: breadth-first and depth-first

Our strategy above was somewhat **ad hoc**: we said we would mark some things and then pick up the smallest thing not yet explored, but we did not say systematically how to do that. To get an actual computational procedure we must elaborate the strategy carefully. It turns out that there are two broad strategies.

Strategy	How it explores
Breadth-first	Propagate the marks in **layers**: first everything reachable in one step, then everything reachable in two steps, and so on.
Depth-first	There may be three places I can go to, but instead of trying the second, I go to the **first** and from there keep exploring further — going down that path until I hit a **dead end**. Then I **back up** and take the second direction, and so on, until I run out of directions.



We shall study breadth-first and depth-first search in more detail as we go along.

7 The adjacency list

Before that, let us return to the question of how to represent a graph. The problem with the adjacency matrix is visible at a glance: there is a **large number of** 0s, and 0s convey no information to us — only the 1s are useful. The 0s are non-edges and the 1s are edges, and we are interested only in the edges.

How big is the matrix?

For n vertices the adjacency matrix has size

$$n^2$$

regardless of how many edges there actually are.
In the extreme case this is not wasteful, because a graph *can* have about n^2 edges. For an undirected graph in which every pair of vertices is an edge, the number of *edges* is

$$\frac{n(n-1)}{2},$$

since each edge is counted twice — but the matrix has $n(n-1)$ entries filled, because (i, j) and (j, i) are both recorded even though they represent the same edge. For a directed graph one can have $n(n-1)$ edges outright.
But **typically** a graph will have far fewer than n^2 edges — and then it is not clear that this large matrix is the best way to represent it.

Definition 3.3 — Adjacency list

In an **adjacency list** we write down, for each vertex, **which vertices are its neighbours**. For each vertex we record exactly what would have been the 1s in that vertex's row of the adjacency matrix.

For the directed airline graph: 0 is connected to 1 and 4, so against 0 we put the list $1, 4$; similarly 5 is connected to 3 and 7, so against 5 we put $3, 7$. Row 5 of the adjacency matrix has two 1s, at columns 3 and 7; instead of writing the 0s in the other eight positions, we just write those two positions.

Adjacency list — the directed airline graph

Vertex	⟶	Out-neighbours
0	⟶	1, 4
1	⟶	2
2	⟶	0
3	⟶	4, 6
4	⟶	0, 3
5	⟶	3, 7
6	⟶	5
7	⟶	4, 8
8	⟶	5, 9
9	⟶	8

Comparing the two representations of the same graph, it is obvious that the adjacency list is typically **much more compact** in terms of the space it occupies.

8 Trade-offs between the two representations

Of course, we have to do different things when we compute with the two representations.

Task	Adjacency matrix	Adjacency list
Is j a neighbour of i?	look up the single cell $A[i][j]$ — **constant time**	walk down the whole list for i — **more expensive**
List all neighbours of i	scan the entire row: **order** n time, whatever the degree	read off the entries: time **proportional to the degree**
Space	n^2, whatever the number of edges	proportional to the number of edges
Incoming edges (directed)	read the **column** for i	awkward — recorded in *other* vertices' lists

Two of these deserve a comment.

- **Checking a single edge.** In the matrix we just look at the cell (i, j) and see whether it is 1 — assuming every entry can be looked up in the same amount of time, this costs the same for every i and j. In the list, to check whether there is an edge from 8 to 9, we go to 8 and walk down: first we see 5, then we see 9. So we may have to go through the entire list.

- **Collecting all neighbours.** Here the list wins. If there are five neighbours there are five entries, and we read five entries. In the matrix, regardless of how many real neighbours there are, we must scan the **entire row**, because we never know whether another 1 is coming up — we cannot stop early, since a neighbour might sit in the very last position. So we must spend order n time whatever the actual degree, whereas with a list the time at each node is proportional to the **degree**.

Which is better?

In practice many graphs have **small degree** — a given node is not connected to very many others. So a procedure whose cost is proportional to the degree rather than to the number of nodes often runs much faster, and that is why the list representation is useful.
But there are **trade-offs**, and it is not always the case that one is better than the other. Typically an adjacency list saves space, but in some situations it incurs additional computation, and vice versa. The matrix is very simple to work with, at the cost of scanning rows and columns; with a list some things are less convenient — for instance, finding an **incoming** edge in a directed graph, since that edge is recorded in some *other* vertex's list, so one has to go and look there.

9 Summary

We cannot think of a graph merely as a **picture** if we want to describe a procedure on it; we need a **representation** — a way of writing it down that we can manipulate mathematically. We came up with two:

- the **adjacency matrix**, an $n \times n$ matrix for n vertices, with $A[i][j] = 1$ if there is an edge from i to j and 0 otherwise;
- the **adjacency list**, more compact when there are not very many edges, listing for each vertex its neighbours.

With either one we can now do **systematic computation** — as we did, using the matrix, to explore whether a vertex v is reachable from a vertex u.

We will look at more such things: in particular at the two strategies we mentioned but did not carry out in detail — **breadth-first search** and **depth-first search** — and then at other properties of graphs that can be computed using these two representations.

Lecture 4 — Breadth-First Search

Visiting and exploring; the queue; the BFS trace; recovering paths with parents; levels as distances

1 Recalling the representations

We have been looking at the question of **reachability** in a graph. To find whether a target vertex is reachable from a source vertex, we systematically explore the graph: mark the source vertex, go to its neighbours and mark them, and keep doing this until the target becomes marked.

To do this as a *procedure* we needed a way of representing the graph which is not a picture, and we came up with two:

- the **adjacency matrix**, with a row and a column for every vertex (numbered 0 to $n-1$), where the entry at row i, column j is 1 if there is an edge from i to j and 0 otherwise;
- the **adjacency list**, which avoids the wastefulness of all those 0s by recording, for each vertex, only its neighbours.

A worked check on the graph

Look at v_4. It has an edge to v_0 and an edge to v_3 (in the directed picture, both outgoing *and* incoming), and an edge involving v_7 as well. So the neighbours of 4 are

$$\{0,\ 3,\ 7\},$$

and correspondingly row 4 of the adjacency matrix has 1s at positions 0, 3 and 7. When writing out an adjacency list it is easy to leave an entry out by mistake — so it is always worth checking the list against the row.

Having represented the graph in a way we can manipulate, we now come to the second part of the story: **how do we systematically explore it?** Last time our exploration was somewhat ad hoc. There are two systematic ways to do it — **breadth-first** and **depth-first** — and in this lecture we look at breadth-first.

2 Exploring level by level

When we explore a graph breadth-first, we do it **level by level**. We start with a vertex, then identify all of its neighbours — all the edges connecting it to them — and declare those endpoints reachable. Then we go to those neighbours and see what is reachable from there: those are the vertices **two levels away**. And so on.

The danger of going round in circles. Suppose we start at 5 and find its neighbours to be 3, 8 and 7. Going to 3, we find its neighbours to be 4 and 6. Now looking at the neighbours of 6 we find that 5 is a neighbour of 6 — but 5 is where we started!
So we must **record that a vertex has been visited**, and make sure we never visit a vertex twice. Otherwise we would go round and round a triangle or a cycle any number of times.

3 Visited and explored

We need to do the visiting and the exploring **exactly once per vertex**, so we must maintain two distinct pieces of information.

Two statuses

- **Visited:** the vertex has been marked, having been found as the neighbour of something we have already seen.
- **Explored:** we have subsequently gone to that vertex and looked at *its* neighbours.

A vertex is explored only *after* it is visited. A vertex which has been **visited but not explored** represents **pending work**.

We record the visited information as a **function**

$$\texttt{visited} : V \longrightarrow \{\texttt{True}, \texttt{False}\}$$

— in programming terms one may think of it as a dictionary whose keys are 0 to $n-1$, but mathematically it is a function telling us, for each vertex, whether it has been visited. Initially

$$\texttt{visited}(v) = \texttt{False} \qquad \text{for every vertex } v,$$

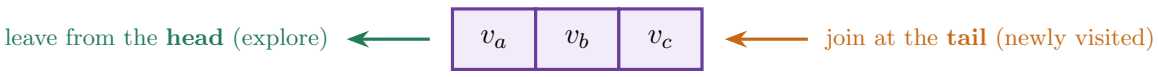
since we have not explored the graph at all.

3.1 The queue

Separately we must keep the vertices which have been marked visited but not yet explored. We keep them in a special kind of sequence called a **queue**.

What a queue is

The word has exactly the meaning you associate with an English queue. Standing in line to buy a ticket, you **join at the end** of the line; the person at the **front** buys a ticket and moves away; the next person moves up to the front. In the same way, as vertices get visited they are added to the end of the queue, and when their turn comes they are explored.



4 The breadth-first search procedure

What exploring a vertex means

To **explore** a vertex i: look at every edge (i, j) in the graph. If j has **not** yet been visited, then

- mark j as visited, and
- add j to the end of the queue of unexplored vertices.

Breadth-first search (BFS) from a source vertex j

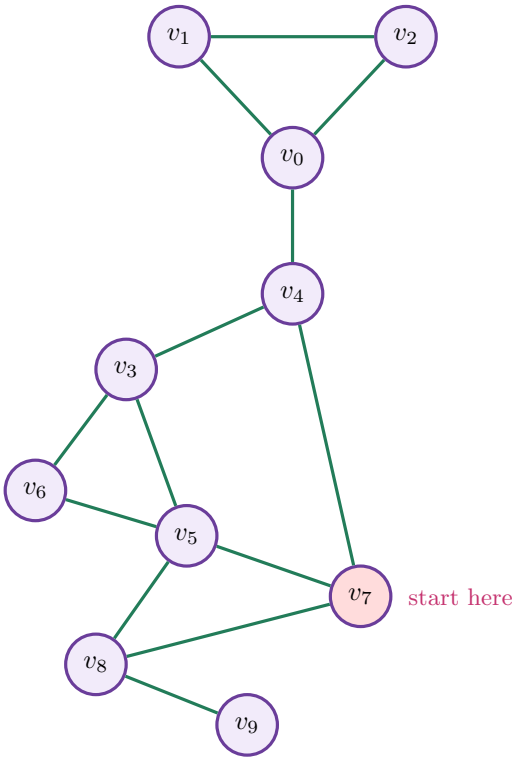
1. Set `visited`(v) = `False` for every vertex v; start with an empty queue.
2. Set `visited`(j) = `True` and **add j to the queue**.
3. **Repeat:** remove the vertex at the **head** of the queue and **explore** it — for each unvisited neighbour, mark it visited and add it to the tail of the queue.
4. **Stop** when the queue becomes **empty**.

Why the empty queue is the right stopping condition

If there is nothing left in the queue to explore, this means every vertex we have visited has also been explored, and there is nothing left to do. That is when BFS terminates.

5 A full BFS trace

Let us explore the following undirected graph on 10 vertices, **starting at vertex** 7. We take the **left** side of the queue as the front (head) and the **right** side as the end (tail). Whenever several vertices are added at once we list them in **ascending order** — the order does not matter, but it is convenient to fix one.



The queue, step by step

Step	Explore	Neighbours	Queue afterwards
—	(mark the source 7)	—	7
1	7	4, 5, 8 — all new	4 5 8
2	4	0, 3, 7 — 7 seen; 0, 3 new	5 8 0 3
3	5	3, 6, 7, 8 — only 6 new	8 0 3 6
4	8	5, 7, 9 — only 9 new	0 3 6 9
5	0	1, 2, 4 — 1, 2 new	3 6 9 1 2
6	3	all already visited	6 9 1 2
7	6	all already visited	9 1 2
8	9	all already visited	1 2
9	1	all already visited	2
10	2	all already visited	**empty — stop**

Reading the trace:

- We mark 7 as visited and put it in the queue. Now the real breadth-first search begins: pick up the first element of the queue and explore it.
- Exploring 7: its neighbours are 4, 5 and 8, so all three get marked and added. Notice that 7 has **gone** from the queue, because it has been explored.
- Exploring 4: its neighbours are 0, 3 and 7, but 7 has already been visited, so we take only 0 and 3. Now 4 has left the queue — but 5 and 8 are still pending from the previous round. So 0 and 3 must **wait their turn**: 4, 5 and 8 are all one level away, and until we finish everything one level away we do not explore anything added at level two.
- Exploring 5: its neighbours include 3, 7 and 8, already visited, so only 6 is new. Exploring 8: its neighbours are 5, 7 and 9, so only 9 is new.
- Now the level-one vertices are finished and we come to 0, which was added at level two. Its three neighbours are 1, 2 and 4; but 4 is already visited — indeed 4 is where we came to 0 *from* — so 1 and 2 get marked.
- At this point **everything** has been marked, and in principle we could stop. But that is not how BFS stops: BFS stops by checking that **every visited vertex has been explored**. So we still explore 3, 6, 9, 1 and 2 in turn; none of them adds anything new, each merely leaving the queue.

When the queue is empty we have run out of work: every vertex visited during the search has been processed. In this graph everything can be reached from 7 by some path or other, so every vertex ends up marked `True`.

6 Recovering the path: parents

We now know, for instance, that we can reach 1 from 7, because `visited`(1) is `True`. But this information tells us **nothing about how** to do it. Knowing that `visited`(j) = `True` after running BFS from i tells us j is reachable from i, but gives us no path.

This can be fixed. How did we get to j? We reached j because we explored j from some vertex k.

Definition 4.1 — Parent

When `visited`(j) is set to `True` while exploring a vertex k, we record

$$\texttt{parent}(j) = k.$$

Following back from j to k through this parent link, k itself must have been added because of some other vertex, so we look at $\texttt{parent}(k)$, say ℓ, then $\texttt{parent}(\ell)$, and so on — everything eventually traces back to the **starting vertex**, which is what marked the first set of vertices visited. In this way we walk backwards and find the reverse path, which we then read out in the forward direction.

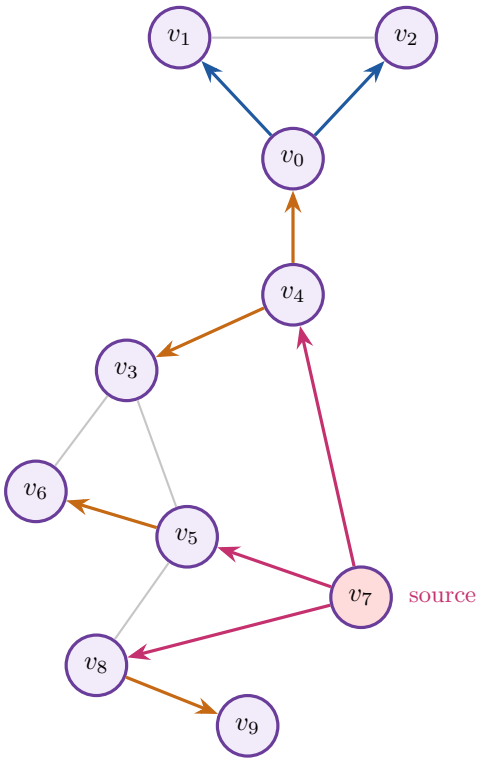
No extra cost

This can be done **along with** breadth-first search at no extra cost — we simply record more information as we go. Note also that the **source vertex has no parent**: 7 got visited because we *initiated* the search there, not because some other vertex marked it. Every other visited vertex has a parent.

The parent recorded for each vertex (BFS from 7)

vertex	0	1	2	3	4	5	6	7	8	9
parent	4	0	0	4	7	7	5	—	7	8

The parent links, drawn on the graph, form the **BFS tree**:



6.1 Reading off two paths

Example 4.1

What is the path from 7 to 6 that breadth-first search discovered?

Go to 6: it says its parent is 5 — "I came from here." And 5 says it came from 7. Reading the trace back in reverse:

$$\boxed{7 \to 5 \to 6}$$

Example 4.2

And the path from 7 to 2?

Vertex 2 says it came from 0; 0 says it came from 4; and 4 says it came from 7. Reading it backwards:

$$7 \to 4 \to 0 \to 2$$

So by keeping this parent information as we explore, we also record **a path** to every reachable vertex from the source.

7 Recording the distance: levels

BFS explores neighbours level by level, so the **level** of a vertex indicates the earliest we can get to it from the source. Everything at level 1 is reachable directly in one edge; anything at level 2 is reachable from level 1 but not directly, so it needs two edges; and so on. The level therefore gives a notion of **distance** from the source.

Definition 4.2 — Level

Replace the `True`/`False` of `visited` by a **level**:

- initialise $\texttt{level}(v) = -1$ for every v — since the minimum level of a reachable vertex is 0, any vertex still at -1 at the end is exactly one whose `visited` would have been `False`, i.e. it was never reached;
- set $\texttt{level}(i) = 0$ for the source i;
- whenever we visit j from k, set

$$\texttt{level}(j) = \texttt{level}(k) + 1.$$

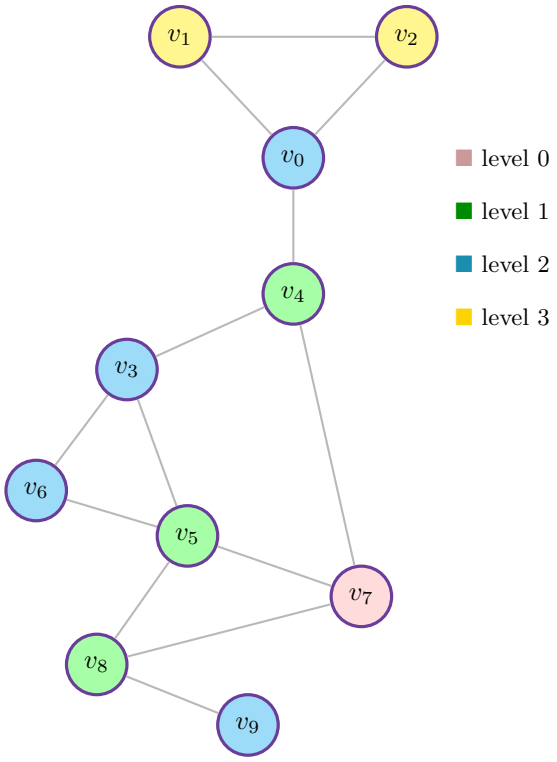
The level of each vertex (BFS from 7)

vertex	0	1	2	3	4	5	6	7	8	9
level	2	3	3	2	1	1	2	0	1	2

Following the computation: 7 starts at level 0. Exploring 7 puts its neighbours 4, 5, 8 at level 1. Exploring 4 gives 0 and 3 level 2, since the level of 4 was 1 and $1 + 1 = 2$.

The order of exploration does not change the level

Although we explore 5 *after* 4, the level of 5 was the *same* as the level of 4 — both were added at the same time, at level 1; we merely happened to process 4 first. So what we reach from 5 is also at level 2: vertex 6 becomes level 2, and likewise 9 from 8. Finally, exploring 0 (at level 2) puts 1 and 2 at level 3, since to reach them we must first go to 0, which was already two edges away.



Levels give shortest distances

Because we proceed level by level, the **length of the shortest path** from i to j, measured in the number of edges travelled, is exactly `level`(j). There may be longer ways to get there, but there is **no shorter** way.

At any point during the run, the vertices which still carry level -1 are precisely those not yet visited. And of course we may keep **only** the level, **only** the parent, or **both** — whichever we want: the parent gives the path, the level gives the shortest distance.

8 Summary

Breadth-first search is a systematic strategy to explore a graph **level by level**. The broad idea is to propagate marks from a source vertex to its neighbours, to their neighbours, and so on — but we need a way of doing this that does not go round and round in cycles and that terminates sensibly. BFS does it by

- recording which vertices have been **visited**, and
- maintaining a **queue** of visited vertices which are yet to be **explored**.

We then saw that we can add **parent** information to recover the path, and maintain **level** information to record the distance — the shortest path in terms of the number of edges.

9 Looking ahead: weighted graphs

In general, a graph may record more information than just the presence of an edge. For an airline network or a railway network, each edge would typically carry a number which we abstractly call a **cost**. That cost could be

- a **real cost** — the price of a ticket from this station or airport to that one;

- a **distance** — how many kilometres the route travels;
- a **time** — the same distance may be covered in different times depending on the quality of the tracks and the number of stops in between;
- or, on a road network, a measure such as how many litres of petrol the journey would consume.

In such graphs — called **weighted graphs** — the shortest path is no longer simply a matter of the number of edges. A single short edge may carry a very high cost: a direct flight can cost much more than one going via an intermediate airport, because the airline is trying to encourage people onto flights to unpopular airports in order to fill the planes. So **the fewest edges is not always the cheapest path**, and we shall explore this separately when we look at weighted graphs.

Lecture 5 — Depth-First Search

Suspending and resuming; the stack; a full DFS trace; DFS numbering; cut vertices and bridges

1 The other systematic strategy

We said there are two systematic ways to explore a graph. We have looked at **breadth-first search**, which explores a graph **level by level**, using a **queue** to hold the vertices that have been visited but not yet explored. The other strategy is **depth-first search**.

The idea of depth-first search

Start from a vertex i and pick **any one** of its neighbours j that has not been explored. (In breadth-first search we picked *all* its neighbours and put them into a queue; here we pick just one.) Now **suspend** the exploration of i and start exploring j instead.
At j we look at its neighbours, again pick one that has not been explored, suspend j, and go on — and we keep doing this until we run out of vertices we can reach down this path.
When we reach a vertex that cannot be explored any further, we **come back along the path we took** and find the first point at which there was another unexplored choice — that is, we **backtrack to the nearest suspended vertex that still has an unexplored neighbour** — and explore that neighbour, and so on.

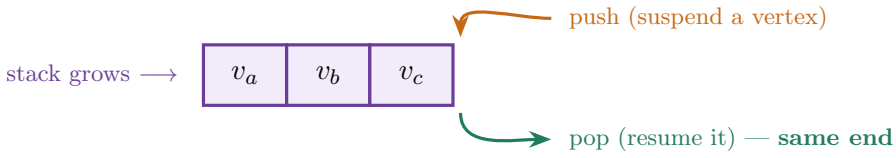
2 Queue versus stack

In breadth-first search we kept track of the visited vertices level by level, put them into a queue, and processed that queue in a **first in, first out** manner: the vertices added at level 1 get processed before those at level 2, which get processed before those at level 3, and so on.

In depth-first search, having walked down some distance, we come back to the point where we last stopped and see whether there was something else we could do from there. So what we need is not first in, first out but **last in, first out**: *the last vertex suspended is the first that is restored and checked again.* Such a structure is called a **stack**.

Structure	Discipline	Used by
Queue	first in, first out — join at the tail, leave from the head	breadth-first search
Stack	last in, first out — add at one end, remove from the *same* end	depth-first search

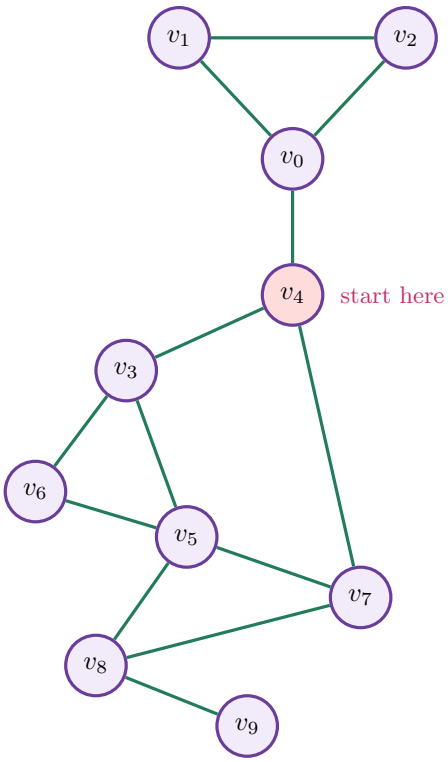
These are two very fundamental ways of organising sequences.



For the trace below, the stack — like our queue — **grows from left to right**: when we add something we put it on the right. But unlike a queue, when we remove something we also remove it from the **right**, not from the left.

3 A full DFS trace

For a change, instead of starting at 7, let us run depth-first search from vertex **4** on the same undirected graph. Whenever there is a choice of unexplored neighbour we pick the **smallest**.



The stack, step by step

Action	At	Unexplored neighbours	Stack afterwards
mark	4	0, 3, 7	(empty)
suspend 4, go to 0	0	1, 2	4
suspend 0, go to 1	1	2	4 0
suspend 1, go to 2	2	none	4 0 1
backtrack to 1	1	none left	4 0
backtrack to 0	0	none left	4
backtrack to 4	4	3 still left	(empty)
suspend 4, go to 3	3	5, 6	4
suspend 3, go to 5	5	6, 7, 8	4 3
suspend 5, go to 6	6	none	4 3 5
backtrack to 5	5	7, 8 still left	4 3
suspend 5, go to 7	7	8	4 3 5
suspend 7, go to 8	8	9	4 3 5 7
suspend 8, go to 9	9	none	4 3 5 7 8
backtrack $9 \to 8 \to 7 \to 5 \to 3 \to 4$	—	nothing left anywhere	**empty — stop**

Following the trace in words:

- We mark 4 as visited. Its neighbours are 0, 3 and 7; we may pick any of them, so let us pick the smallest. We suspend 4 and start exploring 0 instead — so 4 goes onto the stack.

- At 0 we suspend and pick one of its unexplored neighbours, say 1; so 0 comes on top of 4 in the stack. Exploring 1, its only unexplored neighbour is 2, so we suspend 1 and go to 2.
- At 2 there is nothing left: its only neighbours are 0 and 1, both already visited. So we begin **moving back**. We backtrack from 2 to the most recently suspended vertex, 1 — but 1 has nothing more to do, having visited both its neighbours. So we backtrack to 0.
- Does 0 have anything more? Note that from 0 we visited 1, and from 1 we visited 2. So although 2 had not been seen when we first arrived at 0, by the time we backtrack to 0 it has been visited. So 0 has no unvisited neighbours either, and we backtrack from 0 as well, arriving back at 4.
- Now 4 says: yes, I have another vertex to visit — namely 3. So we suspend 4 again and explore 3; that whole first section is finished.
- From 3 there are two unexplored neighbours, 5 and 6. Suspend 3 and explore 5; from 5 there are unexplored neighbours 6, 7 and 8, so suspend 5 and explore 6. At this point 4 triggered 3, which triggered 5, which triggered 6 — so 4, 3 and 5 sit on the stack.
- 6 has nothing new, since both 3 and 5 are visited, so we backtrack to 5. Now 5 *does* have more to do — it has 7 and 8. So we suspend 5 **again** (the first suspension was to explore 6; this one is to explore 7) and go to 7. Then 7 has neighbour 8, so suspend 7 and explore 8; and 8 has neighbour 9, so suspend 8 and explore 9.
- At this point every vertex has been marked `True` — but there is still a long stack of suspended vertices, and we must make sure nothing is missed. So: 9 has nothing left (its only neighbour 8 is where it came from); back to 8, nothing more (5 and 7 are marked); back to 7, which was triggered by 5; back to 5, triggered by 3; back to 3, triggered by 4; and back to 4, which by now has nothing left either — from 4 we explored 0, and indirectly through this second branch 7 has already been reached. The stack is empty, so 4 is terminated and we quit.

An everyday analogy

Depth-first search works rather the way you browse the internet: you start reading something interesting, then click a link because it looks interesting, follow that, and before you know where you are you have wandered somewhere far away from where you began. Then you have to follow those links back to where you started and continue. Find the first interesting vertex and go there; suspend, find the next interesting vertex, go there; and so on.

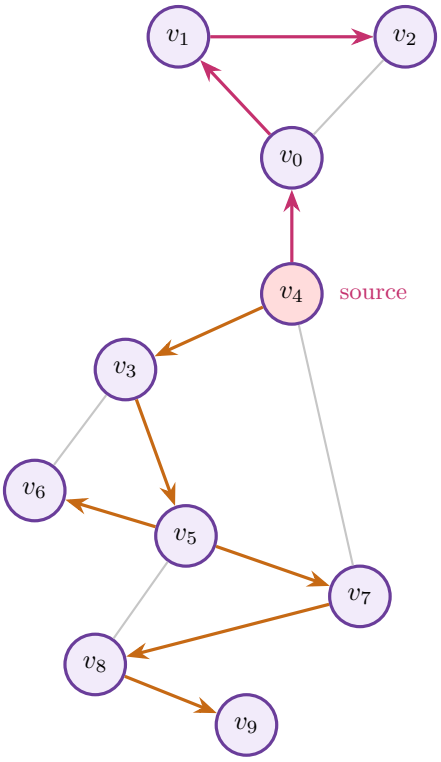
4 The DFS tree and its long paths

Just as with breadth-first search, we could keep **parent** information — recording, for every vertex marked visited, *why* it was marked. In this run:

4 marked 3, 3 marked 5, 5 marked 6, then, on returning, 5 marked 7.

The parent recorded for each vertex (DFS from 4)

vertex	0	1	2	3	4	5	6	7	8	9
parent	4	0	1	4	—	3	5	5	7	8



So depth-first search finds these **long paths**. Reading the parents back from 7: the parent of 7 is 5, the parent of 5 is 3, and the parent of 3 is 4, giving

$$4 \to 3 \to 5 \to 7.$$

> But this is **obviously not the shortest path** — 4 and 7 are joined by an edge, so the shortest path has just one edge. Depth-first search does **not** do what breadth-first search does in terms of shortest paths.

It may then seem strange that we use this indirect way of exploring when we have something apparently better. It turns out that depth-first search is very useful for **other** things.

5 DFS numbering

One thing we can do in depth-first search is keep track of the *order* in which vertices are visited. Informally, we keep a **counter**, incremented every time we **enter** a vertex for the first time and every time we **finish** a vertex — finishing meaning that all its edges, and hence all its neighbours, have been visited.

- When the counter was 0 we entered 4; from 4 we entered 0, so the counter became 1; from 0 we entered 1 at counter 2; and from 1 we entered 2 at counter 3.
- There is nothing to do at 2, so we *finish* 2 at counter 4; we finish 1 at 5; we finish 0 at 6.
- Coming back to 4, we enter 3 at counter 7 — and so on.

DFS numbering for the run above

vertex	4	0	1	2	3	5	6	7	8	9
enter	0	1	2	3	7	8	9	11	12	13
finish	19	6	5	4	18	17	10	16	15	14

What the numbers tell us

The counter values carry information. Vertex 1 was visited *after* vertex 0, because 1 started at 2 while 0 started at 1; and 0 finished *after* 1, because 1 finished at 5 while 0 finished at 6. So the interval $[2, 5]$ for vertex 1 sits **inside** the interval $[1, 6]$ for vertex 0 — which is exactly the record that 1 was explored during the exploration of 0.

This **DFS numbering** is very useful, and can be used to find many interesting things. (We shall set it up formally later; here it is only informal.)

6 Special vertices and edges

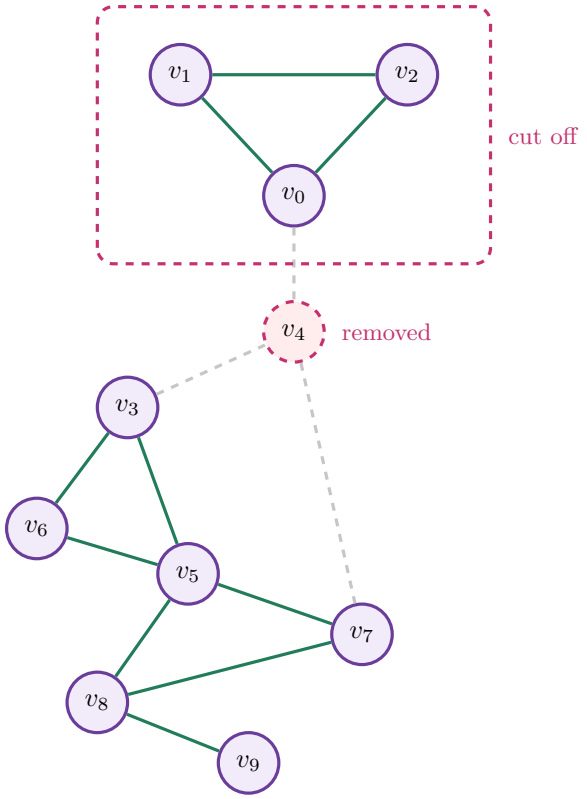
Earlier we talked about problems on graphs such as colouring and matching. But sometimes we also want to find **special vertices**.

6.1 Cut vertices

Definition 5.1 — Cut vertex

A **cut vertex** — also called an **articulation point** — is a vertex whose removal breaks the graph into disconnected portions.

Imagine an electrical network with one power station so critical that if anything happens to it, the entire network splits into two disconnected portions. Or, in our airline network, suppose an airport is knocked out of service by a cyclone.



If the airport at city 4 is knocked out, there is no way to get from the vertices above to those below. We can still travel among $3, 5, 6, 7, 8, 9$, and we can still travel among $0, 1, 2$ — but we cannot travel from the $\{0, 1, 2\}$ section to the rest. The graph has become **disconnected**, so 4 is a cut vertex.

By contrast:

- 3 is **not** a cut vertex: removing it, the graph stays connected by the outer path.
- 5 is **not** one either: we can still get from 4 to 6 the other way round, and reach everything else.

Check for yourself

In this graph, 4 is not the only cut vertex — removing 0 isolates $\{1, 2\}$, and removing 8 isolates 9. It is worth testing each vertex in turn and seeing which removals break the graph apart.

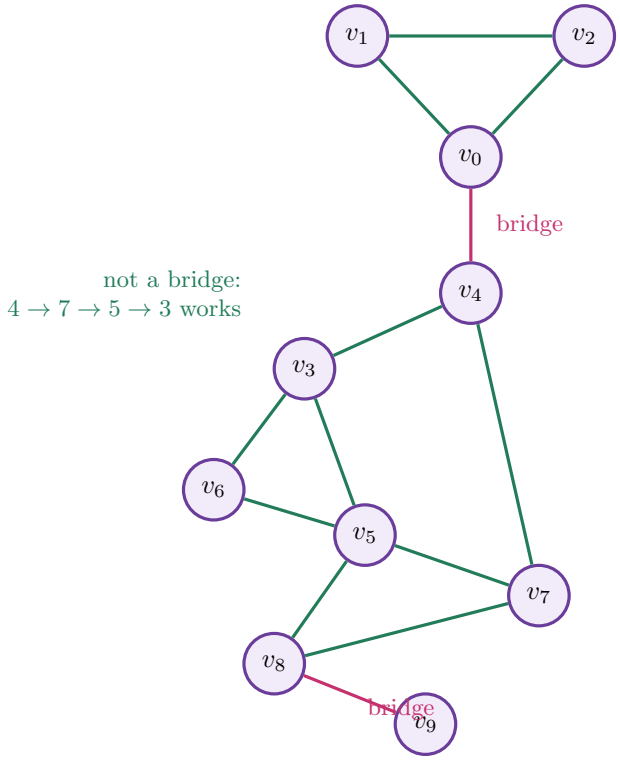
6.2 Bridges

A related question, for edges rather than vertices: are there edges which are **critical**? If there is a route we cannot follow, will that disconnect things?

Definition 5.2 — Bridge

A **bridge** is an edge whose removal disconnects the graph.

The route from 4 to 3 is **not** critical: if we cannot fly 4 to 3 directly we can still follow an indirect path and get there. But if we knock off the route from 4 to 0, then $\{0, 1, 2\}$ is disconnected from the rest of the network — so **the edge** 4–0 **is a bridge**. (Check that the edge 8–9 is a bridge too.)



These kinds of properties of graphs — **cut vertices**, **bridges** and many other interesting things — can be discovered as a **by-product of depth-first search**. You run a depth-first search, do the DFS numbering, and from that numbering work out these structural properties. That is why depth-first search, although it does not find shortest paths, is valuable: it uncovers interesting **structure** within a graph.

7 Summary

Depth-first search is another systematic strategy to explore a graph. We start from a vertex, **suspend** it and go to an unvisited neighbour, suspend that and go to one of *its* unvisited neighbours, and so on. To keep track of the suspended vertices and resume them systematically we use a **stack** — a last in, first out structure — so that when a branch terminates we come back to the most recently suspended vertex.
Keeping track of the sequence in which vertices are entered and finished gives the **DFS numbering**, from which we can uncover structural properties of graphs such as **cut vertices** and **bridges**.

	Breadth-first search	Depth-first search
Structure used	queue (first in, first out)	stack (last in, first out)
How it moves	level by level, all neighbours at once	one neighbour at a time, as deep as possible
Paths found	**shortest** paths (levels give distances)	long paths — not shortest
Chiefly useful for	reachability and shortest distance	structural properties: cut vertices, bridges

Lecture 6 — Components, Cycles, Trees, Directed Graphs, Complexity, DAGs and Topological Sorting

Connected components via BFS/DFS; cycles and simple cycles; trees and forests; DFS numbering and non-tree edges

1 Beyond reachability

We have looked at breadth-first search and depth-first search as two ways to systematically explore a graph. So far all we have done is: starting from a vertex, find out what we can reach.

- **BFS** works **level by level** — we start at a vertex, go to its nearest neighbours, then from those to the next set of neighbours, and so on. Because it processes the graph layer by layer, it discovers the **shortest** path, or shortest level, for every reachable vertex: everything reachable in one edge, then in two edges, then three. The **queue** is what guarantees this order — everything at level 1 enters the queue and gets processed before anything at level 2.
- **DFS** is a very different, rather **aggressive** strategy: the moment it moves to a neighbour it suspends the current vertex and starts exploring the neighbour. It is a bit like looking up information on the internet — you start reading something, find an interesting link before you finish, go to that link, find another, and eventually you must remember to come back to where you were reading in the first place. DFS keeps a **stack** of suspended vertices.

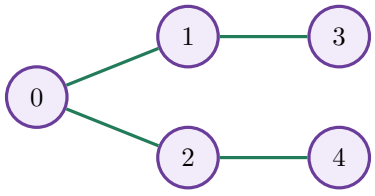
The question for this lecture: **is BFS and DFS only for reachability, or are there more interesting things we can do?**

2 Connected components

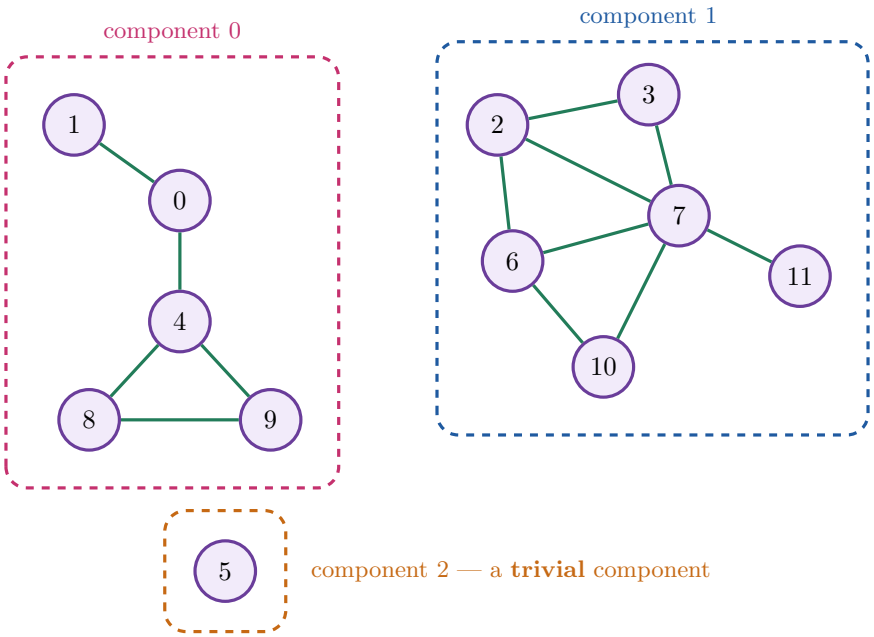
Definition 6.1 — Connected

An **undirected** graph is **connected** if every vertex is reachable from every other vertex.

The first graph below is clearly connected: from any of $0, 1, 2, 3, 4$ you can go to every other vertex. The second graph is **disconnected** — there is a left-hand portion, a right-hand portion, and in addition vertex 5 sits isolated from everything else.



connected — and, as we shall see, a **tree**



Inside each bordered region, everything is reachable from everything else within that region; but there is no way to travel between regions.

Trivial components

Recall that technically there are **no self-edges** in our graphs — we do not assume there is an edge from i to i. So when we call a single vertex a component, we are simply saying that it cannot reach anything, and there is nothing else connected to it that could come back to it. These are the **trivial** components.

3 Finding the components with BFS or DFS

Our goal is to see how BFS or DFS can identify these components. Just as in BFS we kept track of the level number and the parent, and in DFS the parent, we now keep track of **one more quantity** – a **component number** attached to each vertex, labelling them component 0, component 1, component 2, and so on.

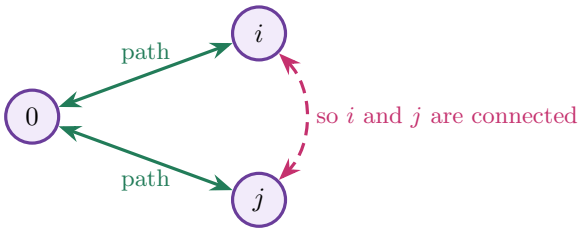
Discovering the components

1. Initialise the component number to 0 and start a BFS *or* a DFS from vertex 0 — it does not matter which. While performing the scan, assign `component`$(v) = 0$ to every vertex reached, exactly as we assign `visited`(v).
2. When that scan finishes, look at the vertices **not yet marked visited**. If there are none, the graph was connected and we are done.
3. Otherwise pick any one of them — say the **smallest** — **increment** the component number, and start a fresh BFS or DFS from there.
4. **Repeat** until every vertex has been visited.

3.1 Why one scan gives exactly one component

Everything found in a single scan of BFS or DFS forms a connected component. The reason is this: if we can reach i from 0 and we can reach j from 0, then since the edges are **undirected** we

can always travel backwards, so we can go from j back to 0 and then on to i. Hence there is a connection from i to j as well.



3.2 Running it on the disconnected graph

For the second graph above:

Scan	Start at	Vertices reached	Component number
1	0	0, 1, 4, 8, 9	0
2	2	2, 3, 6, 7, 10, 11	1
3	5	5 only — stops at once	2

In the first graph there is nothing to do after the first scan, since everything has already been visited. In the second, vertex 2 is a candidate to start a new search because it was not visited from 0 — and since it must be a *different* component, we increment the component number before starting. After two scans we have components 0 and 1, but vertex 5 is still unvisited, so we repeat once more; because we cannot go anywhere from 5, that search stops rapidly and gives component 2.

So at the end we know, for example, that 9 and 4 are both in component 0 — they are in the same component — whereas 9 and 10 have component numbers 0 and 1, so they are **not** connected. The point is that we can then look at any two vertices and decide whether they are connected **without** having to start a fresh search from them.

4 Cycles

Related to BFS and DFS is the idea of a **cycle**.

Definition 6.2 — Cycle

A **cycle** in a graph is a path which **starts at some vertex and comes back to that same vertex**.
Recall that technically a *path* does not repeat vertices, so strictly speaking this is a **walk** — we start at a vertex, traverse some edges, and return to the same vertex.

Example 6.1 — two cycles

In the disconnected graph above:

$$4 \to 8 \to 9 \to 4$$

is a cycle. A more complicated one is

$$2 \to 3 \to 7 \to 10 \to 6 \to 7 \to 2.$$

In the second walk, not only are the starting and ending points both 2, but 7 is also repeated along the way. This is still called a cycle.

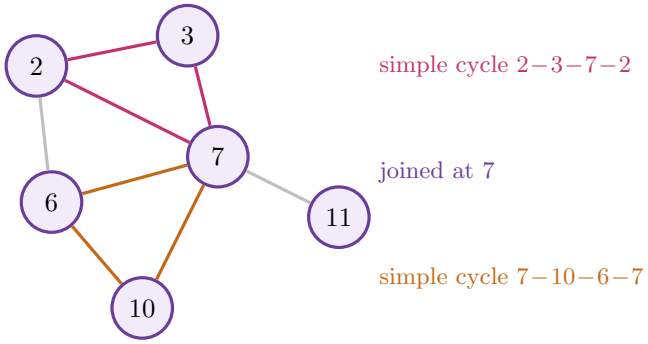
Vertices may repeat, but edges may not. We cannot claim that going from 2 along an edge and immediately coming back is a cycle. A cycle **cannot go back and forth along the same edge** — otherwise every single edge would become a cycle, which is not what we intend.

Definition 6.3 — Simple cycle

A **simple cycle** is one in which the **only** vertex seen twice is the starting vertex, when we close the cycle.

So $4 \to 8 \to 9 \to 4$ is a **simple** cycle. The other one is **not** simple, because we went down, came back up and visited 7 twice — it is really **two simple cycles joined at** 7:

$$2 \to 3 \to 7 \to 2 \qquad \text{and} \qquad 7 \to 10 \to 6 \to 7.$$



Definition 6.4 — Acyclic

If a graph has **no cycles** at all, it is called **acyclic**.

The first graph is acyclic; the second is not. (We speak only of whether a graph *is* or *is not* acyclic.) Our goal now: **if a graph has cycles, how do we find them?**

5 Trees

Definition 6.5 — Tree

A **tree** is a **minimally connected** graph.

Look again at the first, acyclic graph. It is connected — it forms one connected component — and it is acyclic. Moreover, if we drop *any* edge, say the edge 0–2, the graph becomes disconnected. So to be minimally connected we need at least that many edges: here there are five vertices, 0 to 4, and we drew **four** edges.

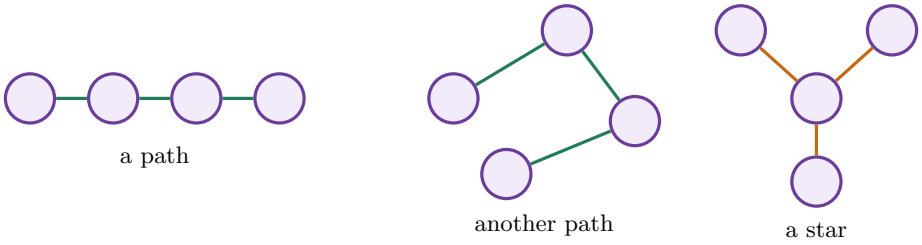
5.1 Facts about trees

A tree on n vertices has $n-1$ edges

It does not matter *which* edges — we may connect them in many different ways — but whichever way we do it, exactly $n-1$ edges are used.

Example 6.2 — four vertices, three edges

With four vertices, one way to make a tree is a **single path**; there are several such paths. A quite different way is to connect all of them to **one central vertex**. Every one of these ways of connecting four vertices minimally uses exactly **three** edges.



A tree is necessarily acyclic

Think of a tree as a minimally connected graph. If it *had* a cycle, then for every pair of vertices on that cycle there would be **two** ways of travelling between them — two ways from 2 to 1, two ways from 0 to 4, and so on. So we could **delete one edge** along the cycle and still have a connected graph, going round the cycle the other way. Hence it would not have been minimal.

Equivalent ways of thinking about a tree

- a **minimally connected** undirected graph;
- a **connected acyclic** undirected graph;
- a **connected** graph with exactly $n-1$ **edges** — and conversely, if it is a tree it has $n-1$ edges.

These are all different ways of describing the same thing, and all are worth remembering.

6 BFS trees and forests

When we explore using BFS, the edges we *use* to visit new vertices — we start at a vertex and visit a neighbour if that neighbour has not been visited, and count that edge as used — actually form a **tree**.

For the acyclic graph at the top, BFS visits *all* the edges, because the graph itself is already a tree and there is no smaller set of edges forming one.

For a graph with multiple components, we must start BFS afresh in each component. So it is not *one* tree — a single tree would connect the entire graph, but the entire graph is not connected. Instead, **each component gets connected by a tree**.

Definition 6.6 — Forest

Just as in English a **forest** is a collection of trees, here too a collection of trees is called a forest. Technically, what BFS discovers is a **forest** inside the graph — one tree per component.

7 Detecting cycles: non-tree edges

Now back to our question — how do we detect whether a graph is acyclic?

We saw that when the graph is acyclic, the BFS tree covers **all** the edges. So if the tree does *not* cover all the edges — if there are **non-tree edges** — then adding any one of them must create a **cycle**.

We can detect a cycle by **searching for non-tree edges**.

How to recover the tree edges

As we go along we can mark edges as visited, just as we mark vertices; or we can look back afterwards at which edges we used. In fact the **parent** information already gives it: if $\texttt{parent}(i) = j$, that means we went from j to i in order to visit i, so ji was a **tree edge**. Either way we can recover the tree edges at the end of the search, and any non-tree edge that exists must form a cycle.

For the disconnected graph, the non-tree edges are

$$6-7, \qquad 3-7, \qquad 7-10, \qquad 8-9,$$

none of which is part of the BFS tree we constructed. Since there is at least one such edge there must be a cycle — in fact here **both** components have a cycle.

8 DFS numbering

We could do all of this with DFS in the same way, but DFS comes with a different strategy for keeping track: **DFS numbering**. Let us build a DFS tree for the same graph and describe the numbering formally through the example.

Definition 6.7 — Pre and post numbers

Maintain a single **counter**, starting at 0. Increment it **every time we enter** a vertex (start exploring it) and **every time we leave** a vertex (finish exploring it). The counter value on entering is assigned to the vertex as its **pre number** (incoming number); the updated counter value on leaving is assigned as its **post number** (outgoing number).

8.1 The trace

Starting at vertex 0 with the counter at 0:

- Enter 0 with pre number **0**. Explore $0 \to 1$: enter 1 with pre number **1**.
- At 1 there is nothing unexplored — its only neighbour 0 is already visited — so we leave, incrementing first: 1 gets post number **2**. We come back to 0 on the stack.
- 0 is **not** finished, since it has another unexplored neighbour, 4. So increment and enter 4 with pre number **3**. Notice the counter simply keeps increasing; we did *not* leave 0, so 0 has no post number yet — it gets one only when all its neighbours are finished.
- At 4 there are two unexplored neighbours besides 0; take the smaller, 8, entering with pre number **4**. From 8 we can go to 9, entering with pre number **5**.
- At 9 we are stuck — its only neighbours 4 and 8 are both visited — so 9 exits with post number **6**. Back at 8 there is nothing more, so 8 exits with **7**. Back at 4: we have done 8, and cannot do 9 since 9 was visited through 8, so 4 exits with **8**.

- Back at 0: both neighbours 1 and 4 are done, so 0 exits with **9**. That completes the first component.
- Now go to an unmarked vertex, 2, **continuing the same numbering**: enter 2 at **10**, 3 at **11**, 7 at **12**, 6 at **13**, 10 at **14** — always taking the smallest neighbour — and at 10 we are stuck, so exit 10 with **15**.
- Back at 6: its neighbours 2, 7 and 10 have all been seen, so exit 6 with **16**. Back at 7: it has another neighbour, 11, not yet explored — so we are *not* done, and we enter 11 with **17**. From 11 there is nowhere to go, so exit with **18**; now 7 exits with **19**, and 3 with **20**.
- Finally back at 2: its other neighbours 7 and 6 were already explored through 3, so exit 2 with **21**.
- Vertex 5 remains, so we start a **third** DFS with counter **22**, and exit immediately with **23**.

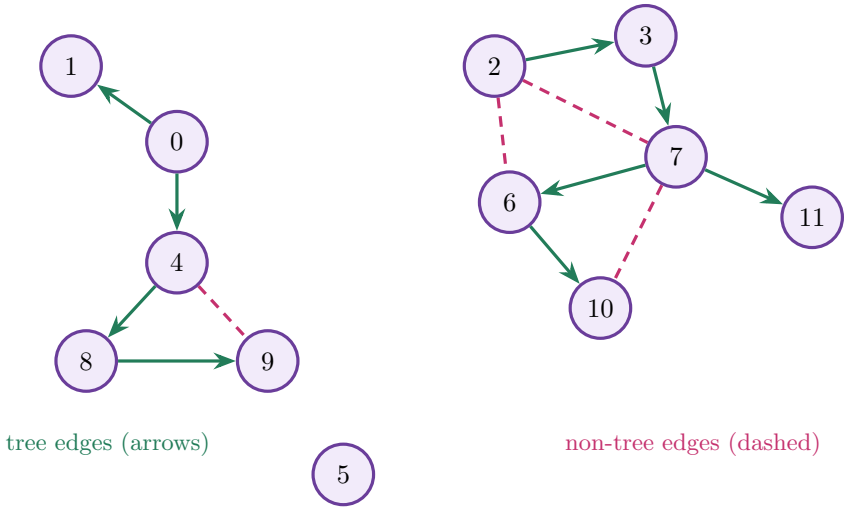
Pre and post numbers for the whole graph

vertex	0	1	4	8	9	2	3	7	6	10	11	5
pre	0	1	3	4	5	10	11	12	13	14	17	22
post	9	2	8	7	6	21	20	19	16	15	18	23

At this point we have not *used* these numbers — we shall see soon why they matter. For now they show that when doing BFS we construct a tree, and when doing DFS we likewise construct a collection of trees, one per component. What the pre and post numbers additionally record is the **order** in which the tree was drawn: how edges were added, and when we backtracked up the tree.

8.2 The DFS tree and its non-tree edges

Ignoring the numbering, some edges did not come into the tree.



- 4–9 is a non-tree edge: our tree went $0 \to 1$, $0 \to 4$, then $4 \to 8$ and $8 \to 9$. Since 9 was covered through 8, we never got to explore 9 from 4.
- 2–7 and 2–6 are non-tree edges, because we never explored them from 2.
- 7–10 is a non-tree edge, because we went $7 \to 6 \to 10$ instead.

Each of these four edges creates a cycle. So, exactly as in BFS: **in an undirected graph, every non-tree edge of a DFS creates a cycle.**

9 Summary

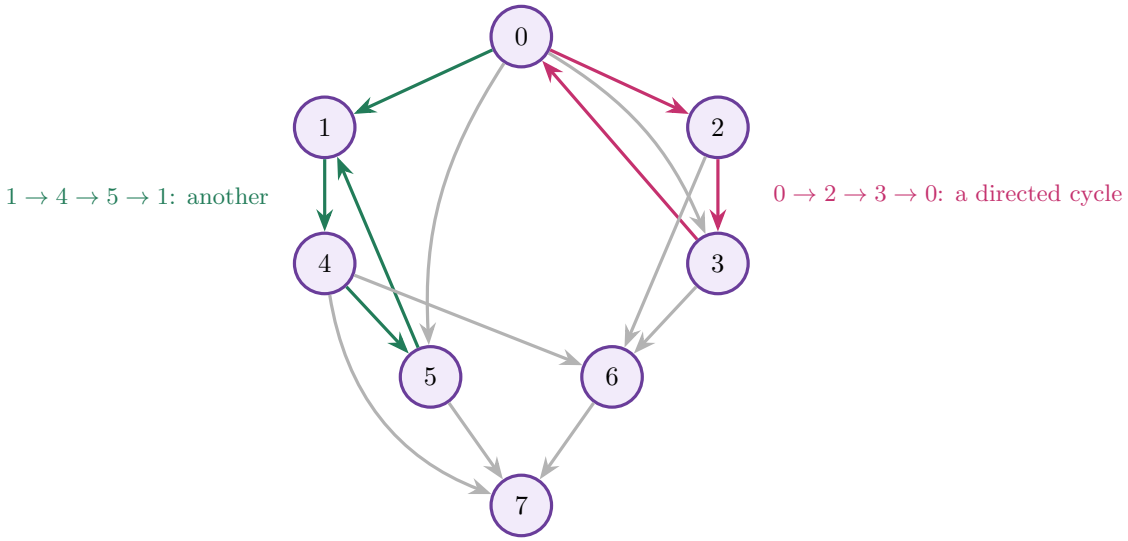
Idea	What we found
Connected component	one BFS/DFS scan discovers exactly one component; repeat from an unvisited vertex, incrementing the component number, until none remain
Cycle	a walk returning to its start; vertices may repeat but edges may not; a **simple** cycle repeats only the start
Tree	minimally connected = connected and acyclic = connected with $n-1$ edges
Forest	what BFS or DFS actually discovers: one tree per component
Non-tree edge	every one creates a cycle — so they are how we detect that a graph is not acyclic
DFS numbering	a pre and a post number per vertex, recording the order in which the tree was built and backtracked

Lecture 7 — Directed Graphs: Cycles and Strong Connectivity

Directed cycles; forward, back and cross edges; classifying them with DFS numbering; strongly connected components; DAGs

1 Cycles in a directed graph

In a directed graph a **cycle is also directed**: we must go around from a vertex, through a set of neighbours, and come back — but **following the same direction** throughout. It is like going around a circle on a **one-way street**: you must follow the one-way system, and you cannot suddenly go the wrong way down it.



Example 7.1 — what is and is not a directed cycle

We can go $0 \to 2$, then $2 \to 3$, then back $3 \to 0$: this **is** a cycle in the directed sense, because we move forwards at every step.
On the other hand, if we try $0 \to 5$, then $5 \to 1$, and then attempt to come back $1 \to 0$, this is **not allowed**: the edge $1 \to 0$ does not exist in that sense. Ignoring the directions there would be a cycle $0-5-1-0$, but **with** directions there is no such cycle.

2 DFS numbering on a directed graph

We again run a DFS and do the DFS numbering. It is **exactly the same** — there is no difference in DFS between the directed and undirected cases — so we follow the same protocol for the pre and post numbers. Start at 0.

- Enter 0 with pre number **0**; enter its first neighbour 1 with pre number **1**; explore 1, entering 4 with pre number **2**.
- From 4 there are several ways to go; the first is 5, entered with pre number **3**. From 5 we go to 7 with pre number **4**.
- Notice that **7 has no outgoing edges at all**, so from 7 we cannot go anywhere: exit 7 with post number **5** and come back to 5.

- Can we do anything more from 5? It had only two outgoing edges — back to 1 and forward to 7. But 1 was already covered and 7 has just been covered, so 5 exits with **6**.
- Back at 4: we explored 5; 7 is no longer available since it was done through 5; but the long edge $4 \to 6$ remains, so enter 6 with pre number **7**. From 6 the only option is 7, already seen, so exit 6 with **8**. Now 4 is done, exiting with **9**; and 1 is done too — it had only the one outgoing edge to 4 — exiting with **10**.
- Back at 0, we now explore the other side: enter 2 with **11**, since we exited the last one with 10. From 2 go to 3 with entry number **12**. From 3 we can only go to 6 or 0, both already visited, so exit 3 with **13**, exit 2 with **14**, and exit 0 with **15**.

Pre and post numbers, DFS from 0

vertex	0	1	4	5	7	6	2	3
pre	0	1	2	3	4	7	11	12
post	15	10	9	6	5	8	14	13

A sanity check

There are 8 vertices here, so there should be $8 \times 2 = 16$ numbers, since we number every **entry** and every **exit**. Starting at 0, we must finish at 15 — and we do. If you do not end at 15, there is a problem.
In general, for a graph with n vertices, a DFS starting the counter at 0 must have final exit number

$$\boxed{2n - 1.}$$

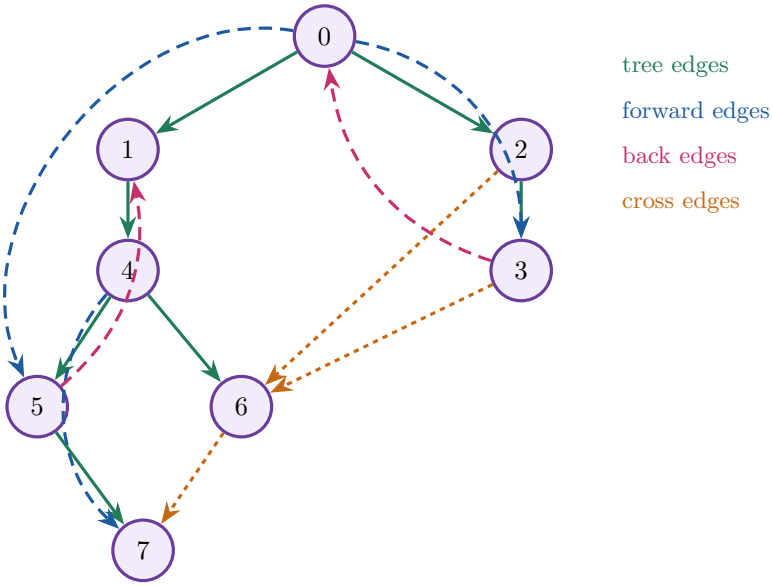
3 Three flavours of non-tree edge

The edges we followed during the DFS are the **tree edges**. This is now a **directed** tree, but otherwise it has the same structure as before — a tree connecting all the vertices we visited.

The **non-tree** edges, however, now come in different flavours.

Definition 7.1 — Forward, back and cross edges

- A **forward edge** follows the direction of the tree, going from a **higher** node to a **lower** node — in the same direction as the path it bypasses. You are short-circuiting: going like a **flyover** on a road, passing over some intersections and landing at a later point.
- A **back edge** is the converse: it goes **up** a path in the tree, from a lower node to a higher node, along a path that already exists.
- A **cross edge** cuts **across** different branches of the tree — neither up nor down.



Forward	Back	Cross
$0 \to 3$	$5 \to 1$	$6 \to 7$
$0 \to 5$	$3 \to 0$	$2 \to 6$
$4 \to 7$		$3 \to 6$

For instance $6 \to 7$: vertex 6 is on one branch and 7 is on another, so we are not going up or down from 6 to 7 — we are going **across**.

4 Which non-tree edges create cycles?

4.1 Forward edges do not

Look at the forward edge $0 \to 5$. The tree edges it corresponds to are

$$0 \to 1, \qquad 1 \to 4, \qquad 4 \to 5.$$

Adding this new edge to the existing path did **not** create a cycle, because the new edge runs in the *same* direction as the path it duplicates — to close a cycle we would need to come back the other way.

4.2 Back edges do

If we go $1 \to 4 \to 5$ and then find a back edge $5 \to 1$, there **is** a cycle. Similarly, if we go $0 \to 2 \to 3$ and then find the back edge $3 \to 0$, there is a cycle.

4.3 Cross edges do not

Consider the cross edge $6 \to 7$. We have the path $0 \to 1 \to 4 \to 5 \to 7$ on one side, and $0 \to 2 \to 6$ (or $0 \to 1 \to 4 \to 6$) on the other. These two paths run *away* from their common origin, so no matter how we connect them — left to right or right to left — we cannot form a cycle, because the paths themselves are going in the wrong direction to close one.

Forward edges do **not** create cycles. Cross edges do **not** create cycles.
Only **back edges** create cycles.

In the undirected case it was very simple — every non-tree edge indicated a cycle. In the directed case it is more subtle: it is not enough to find a non-tree edge, we must find a non-tree **back** edge. So the problem becomes one of **classifying** the non-tree edges.

5 Classifying edges with the pre and post numbers

This is the **first place where we actually use** the pre and post numbers. Think of each vertex as owning an **interval** $[\texttt{pre}, \texttt{post}]$ — the stretch of counter values during which we were processing that vertex and its neighbours.

Classifying a non-tree edge $u \to v$

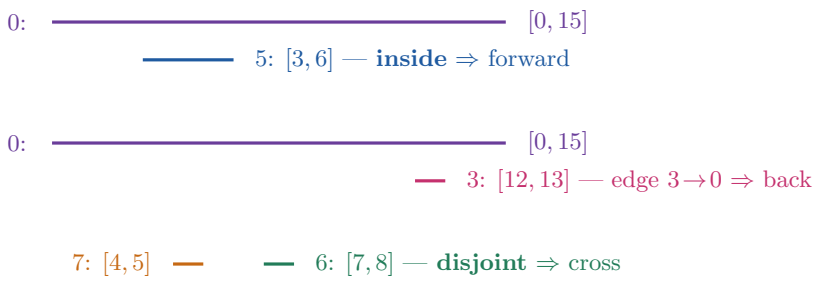
Forward:	$[\texttt{pre}(v), \texttt{post}(v)] \subset [\texttt{pre}(u), \texttt{post}(u)]$	ending interval inside the starting one
Back:	$[\texttt{pre}(u), \texttt{post}(u)] \subset [\texttt{pre}(v), \texttt{post}(v)]$	starting interval inside the ending one
Cross:	the two intervals are **disjoint**	

5.1 Reading the three cases

Forward, $0 \to 5$. Vertex 5 has interval $[3, 6]$ — we entered it at 3, went to its neighbours, and finished at 6. Vertex 0 has interval $[0, 15]$: we started processing 0 when the counter was 0 and finished when it was 15. So the *entire* processing of 5 happened before we finished 0; vertex 5 was processed as **part of** the processing of 0. The edge goes from the **bigger** interval to the **smaller** one, so it is a forward edge.

Back, $3 \to 0$. Here we go from 3, with interval $[12, 13]$, back to 0, with interval $[0, 15]$. Again this says 3 was processed while we were processing 0 — but now the edge is *reversed*, so the **starting** interval is contained in the **ending** interval. That is a back edge.

Cross, $6 \to 7$. Here the intervals are $[7, 8]$ and $[4, 5]$ — **disjoint**. We finished processing one path completely and then went back and started another; by the time we started on the second path all the numbers of the first were exhausted, so the *out* of one happened before the *in* of the other.



So this is how the pre and post numbers let us discover which non-tree edges are back edges, and therefore decide whether a directed graph has a cycle at all.

6 Strong connectivity

Just as cycles must take direction into account, so must the notion of **connectivity**. In an undirected graph we said a graph is connected if every vertex can be reached from every other vertex. In a directed graph we must ask whether we can go *following the directions*.

Definition 7.2 — Strongly connected

A pair of vertices i and j are **strongly connected** if we can go from i to j **and** come back from j to i — possibly by a different path.

If they are not strongly connected, it is possible to go from i to j but not to come back.

Example 7.2 — vertices 0 and 1

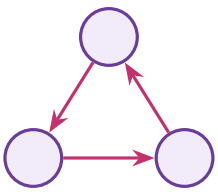
We can go from 0 to 1 directly. But there is **no way** to go from 1 back to 0: from 1 we can only reach 4 and onwards, and the only incoming edge to 0 in this graph comes from 3 — and 3 cannot be reached from 1 by any path.
So 0 and 1 are connected in one direction but not backwards, and therefore they are **not** strongly connected.

Definition 7.3 — Strongly connected component (SCC)

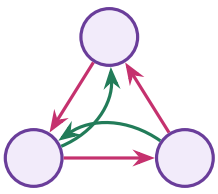
The correct notion of a component for a directed graph is one in which **every pair of vertices is strongly connected** — we can go from everywhere to everywhere and come back, so we can move anywhere within that component without worrying about where we start.

6.1 What an SCC looks like

For three vertices, a strongly connected component is simply a **cycle**: if we have a directed triangle, we can go round it and reach anywhere from anywhere. There could of course be *more* edges — that does not matter — but there must be a **minimum** number of edges so that we can travel in both senses.



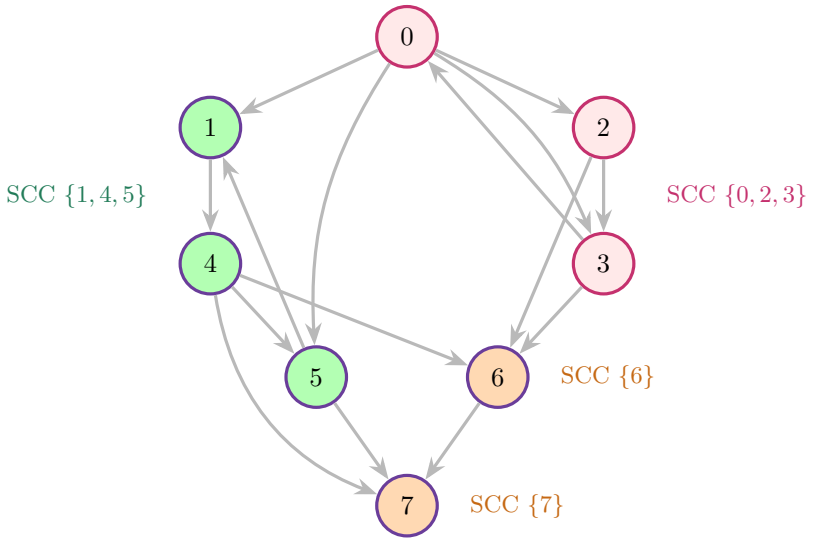
a directed cycle: an SCC



more edges: still an SCC

6.2 The SCCs of our graph

- $\{1, 4, 5\}$ forms a cycle — we can go round in that direction and reach anywhere from anywhere.
- $\{0, 2, 3\}$ likewise forms a cycle.
- **6** and **7** are stuck on their own: if we leave 6 we cannot come back to it by any path, and if we leave 7 we cannot come back — in fact we cannot leave 7 at all, since it has no outgoing edges.



Computing the SCCs

We shall not cover this in detail in this course — perhaps at a later stage — but the same DFS numbering we have just done can **also** be used to compute the strongly connected components. We saw it used to detect back edges and hence cycles; it can equally detect SCCs.

7 Summary

BFS and DFS are the primary strategies for **reachability** in a graph — but we can do much more with them:

- identify the **connected components** of an undirected graph;
- uncover a **tree** of the edges processed during the search — and in an **undirected** graph, any edge outside that tree must form a **cycle**, so the presence of non-tree edges alone proves there is a cycle;
- for a **directed** graph the situation is more complicated: there are three types of non-tree edge — **forward**, **back** and **cross** — and **only back edges generate cycles**. This is where the DFS **pre and post numbering** is used, to detect which non-tree edges are back edges;
- and finally, the notion of a **strongly connected component**, where every pair of vertices is reachable from the other — which the same DFS numbering can also identify.

DFS numbering can be used for many other things too. It identifies the **critical vertices** — a **cut vertex** or **articulation point**, whose destruction disconnects the graph. This matters in, say, a **power network**: if a power station relaying power goes out of action and the network splits so that two parts can no longer supply each other, that station is something we must be extra careful to protect. Similarly there are **cut edges**, or **bridges**, whose removal makes the graph fall apart. Both can be identified using DFS numbering.

8 Looking ahead: directed acyclic graphs

The idea of looking for cycles is particularly important in the **directed** sense, because of a very important class of graphs we shall study next week.

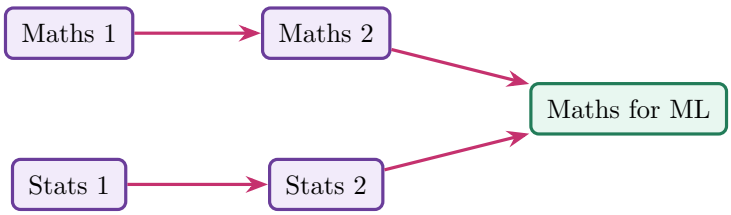
Definition 7.4 — Directed acyclic graph (DAG)

A **directed acyclic graph** is, as the name suggests, a directed graph which has **no cycles** in it.

A DAG is very often used to represent **prerequisites**, **preconditions** or **dependencies**.

Example 7.3 — course prerequisites

Suppose we describe the course contents of this programme: Maths 1 must come before Maths 2, and Stats 1 before Stats 2. There is no dependency between Maths 1 and Stats 1, so those may be done in either order, or one postponed relative to the other — but Maths 2 cannot be done before Maths 1.
So we draw an edge saying M_1 must come before M_2, and another saying S_1 must come before S_2. Now suppose that in the third semester there is a **Maths for ML** course requiring both M_2 and S_2 to be completed.



Now we ask: **in what order can these be done?** Clearly Maths for ML can only be taken after all four earlier courses are finished. But among those four there is flexibility — Stats 1 could be taken after Maths 2, or Maths 1 after Stats 1, and so on.

These kinds of properties — which sequences are compatible with a given set of dependencies — are used very often, and can be worked out by analysing directed acyclic graphs, which we shall do later on.

Lecture 8 — How Long Do BFS and DFS Take?

Counting the work; n^2 versus $n + m$; the sum of the degrees; handshaking; complete and bounded-degree graphs

1 Recalling the two searches

Having seen some of the applications we can achieve using BFS and DFS, let us go back a little and look at the **connection between the searches and the representations** we talked about — adjacency matrices and adjacency lists.

The two strategies, formally

Breadth-first search explores a graph **level by level**. We maintain two pieces of information: the flag `visited`, indicating whether a vertex has been visited, and a **queue** of unexplored vertices. Initially all vertices are unvisited; we start from a vertex j, set `visited`(j) = `True`, add j to the queue, and repeatedly process the queue — take out the first element, look at its neighbours, and for any unvisited neighbour mark it visited and push it into the queue so it gets processed later. When the queue becomes **empty**, BFS terminates.

Depth-first search is a kind of **aggressive** or **impatient** traversal. We start at i and visit one neighbour j; instead of going back to i and continuing with another neighbour of i, we **suspend** i and go to the neighbours of j. The first time we find a neighbour k of j, we suspend j and continue from k. When we can go no further we go **back**, traversing back through the suspended vertices until we find one that is not finished, look for another neighbour, and so on. Here we keep the `visited` information as in BFS, but also a **stack**, remembering the suspended vertices so we can go back in the correct sequence — **last in, first out**.

2 Measuring the size of a graph

What we want to talk about now is **how much time** this takes, as a function of the size of the graph we are trying to explore. In a graph there are two parts: the vertex set, and the edge set, which is a subset of the pairs of vertices — the edge relation is a subset of $V \times V$.

The two parameters

$n = |V|$ = the number of vertices, $m = |E|$ = the number of edges.

On the *same* set of vertices we can draw many edges or few edges. So the number of edges is in a sense an **independent parameter** — a separate measure of how complicated the graph is.

2.1 How small and how large can m be?

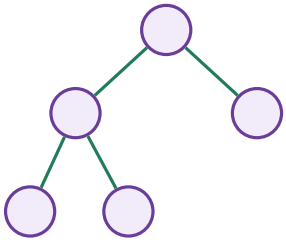
- If we have a **tree** — a minimally connected graph — then we have $n - 1$ edges. So there are interesting graphs in which the number of edges is roughly the same as the number of vertices: n vertices, $n - 1$ edges.

- On the other hand, if we connect **everything to everything**, then for every pair of vertices there is an edge, giving

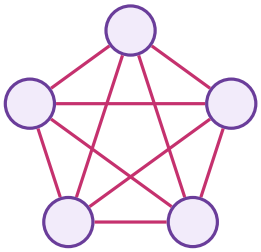
$$\frac{n(n-1)}{2} = \binom{n}{2}$$

edges — which is about n^2.

So m can be of order n, or of order n^2.



a **tree**: $n = 5$ vertices, $m = 4 = n - 1$ edges



a **complete** graph: $n = 5$, $m = \binom{5}{2} = 10$ edges

3 Counting the work done by BFS and DFS

The first thing to note is that both searches **visit and explore every vertex exactly once** — that is the whole purpose of the `visited` flag, which makes sure we never go back to an already-visited vertex and try to visit it a second time. So there are n occasions on which we visit and explore a vertex.

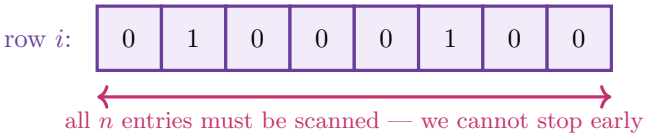
Now, what does **exploring** a vertex mean? It means looking at **all its neighbours** — that is, looking at all the edges outgoing from that vertex. The question is how long that takes when we actually do it *computationally*, rather than by looking at a picture and working by hand.

3.1 With an adjacency matrix

In the adjacency matrix the entries are 0 and 1, and the (i, j)-th entry indicates whether there is an edge from vertex i to vertex j. So if we want the outgoing edges of a vertex i, the only way is to **walk down the entire row** for i:

$$A[i][0], \quad A[i][1], \quad A[i][2], \quad \ldots, \quad A[i][n-1],$$

if the vertices are numbered 0 to $n - 1$.



Why we cannot stop early

Regardless of whether i has many neighbours, few neighbours, or **no** neighbours at all, we must look up the entire row — because a 1 might be sitting in the very last position and we would not know until we had seen it. (In an undirected graph, if we *reached* i during the search then it has at least one neighbour; but a vertex disconnected from where we started — like the

single-vertex component we saw earlier — may have nothing at all.)

So, regardless of how many neighbours i actually has, we spend time proportional to n to discover them. Overall we process n vertices, and for each we scan n entries of the matrix:

Adjacency matrix: BFS or DFS takes time proportional to $n \times n = \boxed{n^2}$

3.2 With an adjacency list

If instead we use an adjacency list, then for each vertex we have an **explicit list** of its neighbours. If it has a lot of neighbours the list is long; if it has very few, the list is short — but we spend **no more and no less** time than we need to scan that list. If a vertex has k neighbours we look at k entries; we do not need to spend n steps looking for k entries when k is small.

The complication is that the **degree** of a vertex — the number of edges incident at it — **varies**: vertex 1 may have a small degree, vertex 2 a big degree, and so on. So to count the steps across all n vertices we must **add up the degrees**:

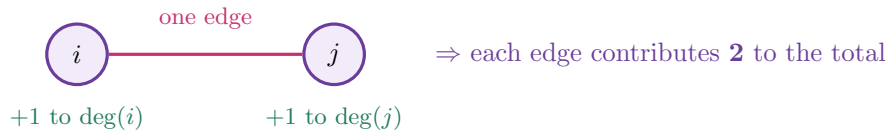
$$\text{total work exploring} = \sum_i \deg(i).$$

So we need a good way of estimating what this sum comes to.

4 The sum of the degrees

What is a degree? It is the number of things going out of a vertex. If i has four edges leaving it, then i contributes 4 to the sum of the degrees. But each of those edges also **terminates** somewhere — say at j. So looking at the edge (i, j):

$$\text{it adds 1 to } \deg(i), \qquad \text{and it adds 1 to } \deg(j).$$



Every number appearing in the degree count must come from *some* edge. Since there are m edges, and each contributes to the degree of both its starting point and its ending point:

The handshaking result

$$\boxed{\sum_{i \in V} \deg(i) = 2m}$$

For each edge, from 1 to m, it contributes $1 + 1$ — so the total is 2 times the number of edges.

4.1 The running time with a list

Now we can see the whole picture. With an adjacency list, BFS or DFS takes n steps to visit each vertex, and across the n vertices it takes the sum of the degrees — that is, $2m$ steps — to explore them all.

Adjacency list: BFS or DFS takes time proportional to $\boxed{n+m}$

Why the n term is still needed

We must visit all the vertices regardless. If the entire graph is disconnected there are no edges at all, so $m = 0$ — but that does not mean the search finishes in zero time: we still have to visit all n things. So we spend n steps looking at all the vertices, and across the vertices the contribution of each is its degree, whose summation is where the m term comes from.

5 Comparing the two: n^2 versus $n + m$

With an adjacency matrix it does not matter what the degrees are or what m is — we end up spending n steps for every vertex, because we cannot find the neighbours of a vertex without looking at the entire row. So the whole distinction is between n^2 and $n + m$.

Size of m	Matrix: n^2	List: $n + m$
$m \approx n$ (a tree)	n^2	$\approx 2n$ — a **big saving**
$m \approx n^2$ (complete)	n^2	$\approx n^2$ — roughly the same

If m is n^2 the two are roughly the same. But if m is small we have a difference between something proportional to n and something proportional to n^2 — and that is why adjacency lists can be beneficial.

So although the adjacency matrix is fine for *understanding* what is going on, mechanically it can cost us — not just because it is a large thing keeping a lot of 0s, but also because to extract the interesting information about **who the neighbours are** we have no better way than to walk down the entire row.

6 More facts about degrees

6.1 Reading the degree off a representation

- From an **adjacency matrix**: just count the number of 1s in the **row** for i. In an **undirected** graph we can equally read it from the **column**, since the number of incoming edges equals the number of outgoing ones — the edge relation is symmetric.
- From an **adjacency list**: the degree is simply the **length of the list** associated with i, since that list holds all the neighbours of i.

6.2 The number of odd-degree vertices is even

We calculated that the sum of the degrees is $2m$. Two times anything must be an **even** number, so the sum of the degrees is even. (The number of edges m itself may well be odd; it is the *sum of the degrees* that must be even.)

Now recall some parity arithmetic:

$$\text{odd} + \text{odd} = \text{even} \quad (3 + 17 = 20), \qquad \text{odd} + \text{even} = \text{odd}.$$

Why, in terms of pairing

If we pair up an odd number of items in twos we get **one leftover** element — that is what makes it odd. Pairing up an even number in twos leaves **no** leftover. So an odd and an even taken together leave one leftover element, hence odd; whereas *two* odd numbers each contribute one leftover element, and those two leftovers pair up with each other, making the total even.

Odd-degree vertices come in pairs

If we spot an **odd-degree** vertex in a graph, it **cannot be the only one** — there must be another, because otherwise the sum of the degrees would become odd (everything else even, plus one odd number, is odd). Likewise, if we find a third odd-degree vertex there must be a fourth.
In other words, the **number of odd-degree vertices must itself be even**. If there were an odd number of vertices with odd degree, the overall sum of the degrees would be odd — which is impossible.

6.3 The range of the degree, and complete graphs

The degree of a vertex can be any number between 0 and $n-1$:

$$0 \;\le\; \deg(i) \;\le\; n-1.$$

Degree 0 happens when the vertex is disconnected from the entire graph; degree $n-1$ happens when it is connected to every one of the remaining $n-1$ vertices. It is not connected to itself, of course, since we assume no self-loops — but it could be connected to everything else.

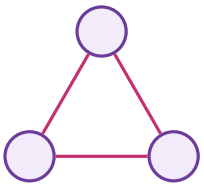
Definition 8.1 — Complete graph

The special case where **every vertex is connected to every other vertex** is called a **complete graph**. In a complete graph

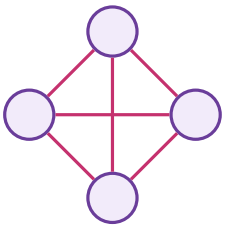
$$\deg(i) = n-1 \text{ for every } i, \qquad m = \frac{n(n-1)}{2} = \binom{n}{2},$$

since every pair is connected.

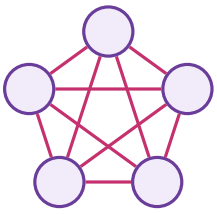
On 3 vertices the complete graph is a **triangle**; on 4 vertices we get a square together with its **diagonals**; on 5 vertices a pentagon with all its diagonals.



$n = 3$: $m = 3$



$n = 4$: $m = 6$



$n = 5$: $m = 10$

7 Bounded-degree graphs

Many graphs we encounter in practical problems — like the ones we discussed for graph colouring or vertex cover — have a degree that is **bounded by an external constraint**, rather than being arbitrarily large.

Example 8.1 — where the bound comes from

- **Security cameras.** Given a particular building, a given intersection will not have more than a certain number of corridors — we cannot have a large number of corridors meeting at one point. So there is an upper bound: no intersection has more than, say, 5 corridors meeting there.
- **Placing ambulances** at an intersection: we want to know how many roads meet at that intersection, and again that is limited.
- **Timetabling**: we want to know how many different courses can be scheduled in the same slot — and if there is a fixed number of courses in the curriculum, not more than that many can be there.

Bounded degree forces a linear number of edges

Suppose every vertex has degree at most k, where k is independent of the size of the graph. Then the sum of the degrees is at most kn, and since that sum is twice the number of edges,

$$m \leq \frac{k\,n}{2}.$$

So the total number of edges cannot be more than **linear** in n.

Therefore, in a bounded-degree graph, you should be working with **adjacency lists**.

8 Directed graphs: in-degree and out-degree

For directed graphs it is no longer enough to talk about *the* degree, because the edges coming in and the edges going out are quite independent of each other. So we must talk about the **in-degree** and the **out-degree**.

Here each edge contributes 1 to an **out**-degree (at its start) and 1 to an **in**-degree (at its end). Hence

$$\sum_i \text{in-deg}(i) = m, \qquad \sum_i \text{out-deg}(i) = m, \qquad \text{and together they still add to } 2m.$$

So the total $2m$ gets **partitioned** into two quantities which add up to m each.



9 Summary

Point	What we found
Adjacency matrix	regardless of how many neighbours a vertex has, we must scan the entire row, so BFS/DFS takes time proportional to $n \times n$
Adjacency list	the time to process a vertex is exactly its number of neighbours, so across the vertices it is the sum of the degrees, giving $n + m$ overall
Variation in m	m can be linear (a tree, $n - 1$ edges) or quadratic (a complete graph, $\binom{n}{2}$ edges) — so it matters which representation we choose
Sum of the degrees	equals $2m$; hence it is even, and the number of odd-degree vertices must be even
Bounded degree	if no node has more than k edges, independent of the graph's size, then $m \leq kn/2$ — necessarily a linear number of edges

So it is important to be able to distinguish these cases and use the appropriate representation — in particular, to **use an adjacency list whenever we can**.

Lecture 9 — Directed Acyclic Graphs and Dependencies

Modelling tasks and dependencies; the office example; topological sorting; longest paths

1 Recalling cycles in directed graphs

Last week we looked at graphs with cycles, and saw that we can use depth-first search and breadth-first search to find cycles in graphs — in particular **directed** cycles. We said that in a directed graph a cycle must follow a **uniform direction**.

Example 9.1 — recalling what counts as a directed cycle

In our directed graph, $0 \to 2 \to 3 \to 0$ **is** a cycle, because we can go from 0 to 2, from 2 to 3, and back to 0.
The other one on the left *looks* like a cycle, but it is not: if we go from 0 to 5 and then 5 to 1, the edge from 0 to 1 is in the **opposite direction**, so we cannot follow it. That is not a directed cycle.

When we run DFS we first construct a **tree** — all the edges traversed during the search are tree edges. Among the **non-tree** edges there are three types:

- **forward edges**, which go forward in the tree;
- **back edges**, which go up the tree from a later node to an ancestor;
- **cross edges**, which go across branches.

Only the **back edges** generate cycles. Using the **DFS numbering** — recording the time at which we enter a vertex to process it, and the time at which we exit after finishing it — we can analyse the tree, look at all the non-tree edges, and decide which category each belongs to. In this way we find all the non-tree edges, and the back edges in particular, which are the cycle-forming ones.

So the question now is: **why were we so worried about cycles in directed graphs to begin with?**

2 A general problem: tasks with dependencies

Let us look at a general problem where we have some **tasks** to do, and there are **dependencies** between the tasks.

Example 9.2 — the startup's new office

A startup is moving into brand new office space. The building has just been constructed — only the bricks are there — and the office must be set up before anyone can move in. A number of things need doing:

- lay the **floor tiles**;
- **plaster** and **paint** the walls;
- lay **pipes** — *conduits*, as they are called — to take wires from here to there. There are wires of two types: **electrical** wires, and **networking** cables for computers, telephone cables and

so on. These cannot go in the same conduit because they interfere, so there are separate conduits for electrical wires and separate conduits for telecom;

- put in the **wiring** itself;
- and after the electrical wiring, put in the **fittings** — the lights, the fans, the switches, and so on.

These are all activities that need to be done, but clearly they **cannot be done in arbitrary order**.

2.1 Where the constraints come from

The reasoning behind each constraint

- **Conduits before tiling and plastering.** The conduits run along the wall to the plug points and to the ceiling, and across the room they travel underneath the floor. So before laying the tiles and before plastering the walls we must put in the conduits — otherwise we would have to break the tiles or break the walls, which is not a good idea.
- **Plaster before paint.** Obviously we must plaster the wall before painting it.
- **Tiling before painting.** We prefer to paint *before* laying the tiles, because the person laying the tiles might mess up the walls with cement. Tiles, on the other hand, are usually washable — so if some paint falls on the tiles it is not a problem.
- **Paint before wiring.** If the wires were hanging out loose during painting, the paint would gum up the wires and get into the cracks. Normally the conduits are sealed up, the walls are painted, and then they are opened again and the wires pushed through.
- **Wiring before fittings.** We cannot put in plug points, lights or fans unless the wires are there.

3 Modelling it as a directed graph

The model

- The **vertices** are the **tasks** we have to perform.
- An **edge** denotes a **dependency**: an edge from t to u says that

$$t \text{ must be finished before } u \text{ can be started.}$$

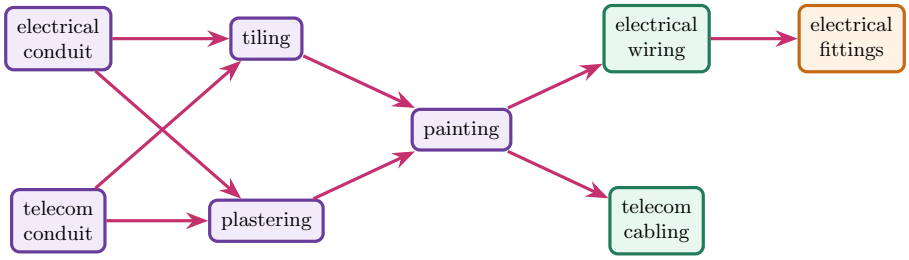
So in our case there is an edge from tiling to painting, because the tiles must be laid before the walls are painted.

The vertices corresponding to our activities are: two different types of conduit (**electrical** and **telecom**), **tiling**, **plastering**, **painting**, two kinds of wiring (**electrical wiring** and **telecom cabling**), and finally the **electrical fittings**.

Now we add the constraints:

- from *each* conduit vertex, an edge to tiling and an edge to plastering;
- from both tiling and plastering, an edge to painting;

- from painting, an edge to each of the wiring and the cabling;
- from the electrical wiring, an edge to the fittings.



This is a **directed graph** constructed from the task constraints given to us.

4 The first question: finding a valid sequence

Our problem is to complete these tasks in a way that **respects the dependencies** — we must make sure the painter comes only after the tiling and the plastering are done.

Here is a possible way. We start with things that have **no dependencies** — the conduits can be laid right at the beginning. Once both are done we can do the tiling and then the plastering. Once both of those are done we can paint; then we can do the wiring, first the electrical wiring and then the telecom cabling; and finally we can put in the electrical fittings:

electrical conduit → telecom conduit → tiling → plastering → painting
→ electrical wiring → telecom cabling → electrical fittings

This is a sequence in which we can complete the tasks so that whenever we come to take up a task, all the tasks that needed to be done before it are already done.

But it is not the only such sequence

- We could have done the **conduiting in the opposite order** — it does not matter whether the telecom conduiting happens before or after the electrical conduiting.
- We could take up the **tiling after the plastering**, because plastering and tiling do not depend on each other.
- We could do the **electrical fittings even before the telecom cabling**, because they go through different conduits and do not interfere with each other.

5 The second question: how fast can it be done?

Suppose we could do things that are independent of each other **at the same time**. We could ask the person laying the tiles to work alongside the one plastering, since tiling and plastering can happen together. Likewise the electrical conduiting and the telecom conduiting could be done in parallel, and so could the wiring and the cabling.

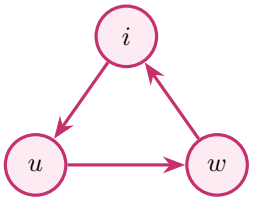
If we optimise so that independent things are done **in parallel**, then **how soon can we finish?** How many days will it take to complete all these tasks, following the dependencies?

6 Why the graph must be acyclic

Formally this is a directed graph — but more importantly it is **acyclic**. There are no cycles in it, because **cycles represent impossible dependencies**: if a depends on b and b depends on a, then which do you do first?

Dependencies cannot form a cycle

If i depends on u, and u depends on somebody else, and that person depends on i, then we all depend on each other, so nobody can get started. If I am waiting for you to finish, and you are waiting for somebody else to finish, and that person is waiting for me — who goes first?
So dependencies cannot form a cycle, and the graph must therefore be a **directed acyclic graph**.



a cycle of dependencies: **nobody can start**

7 Topological sorting

What we are trying to do is find a **schedule** which enumerates the vertices in an order such that the dependencies are respected.

Definition 9.1 — Topological sort

List the vertices in a sequence such that, if a task i must be done before a task j according to the dependencies, then i **appears before** j in the sequence.
Equivalently: for every edge in the graph, the **starting point** of the edge is listed **before** the ending point — indicating that the starting task finished before the second task began.
This problem, on a directed graph, is called **topological sorting**.

8 Longest paths

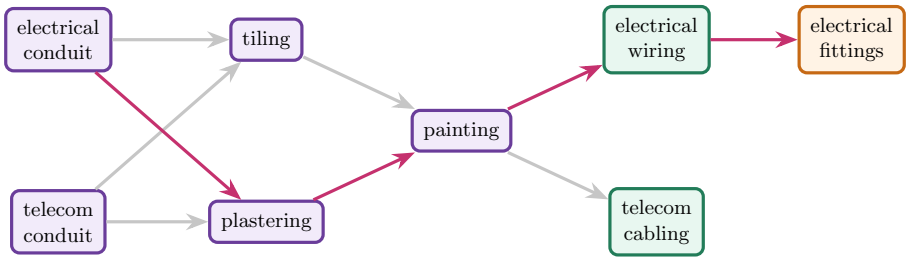
The second thing to discover is **how long we need**. This amounts to finding the **longest path**.

Example 9.3 — two paths through the office graph

Following one route, it takes **four** steps from starting the conduiting to finishing the cabling. But along another path we need to do **five** things in sequence, and we cannot do these any faster:

$$\text{conduiting} \to \text{plastering} \to \text{painting} \to \text{wiring} \to \text{fittings},$$

because plastering can be done only after the conduiting, painting only after the plastering, then the wiring, then the fittings.



the longest path: **5** tasks that must follow one another

So these are the two formal problems we have with DAGs: **topological sorting** and **longest paths**.

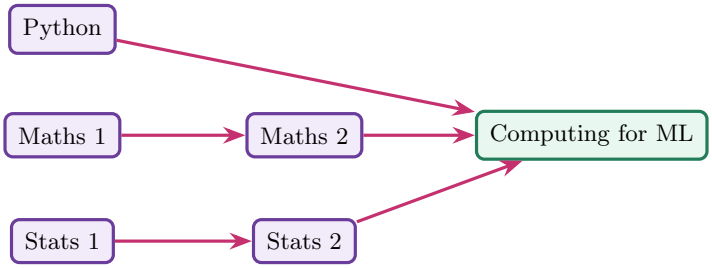
9 Where DAGs arise

Directed acyclic graphs are a **natural way to represent dependencies**. The direction of the edge indicates the direction of the dependency — what must come before what. That the graph is acyclic follows from the fact that a cycle of dependencies would mean nobody could get started.

These arise in many contexts.

Example 9.4 — course prerequisites

A DAG can represent the sequence in which you take courses to complete a degree. Courses usually come with prerequisites: you cannot do Maths 2 before Maths 1, and perhaps you cannot do the Computing for ML course unless you have finished Python programming and both the mathematics courses and both the statistics courses. Given prerequisites like this, find a sequence in which the courses can be taken to complete the degree.



Example 9.5 — cooking

Cooking is a place with a lot of constraints. First we must make sure we have the **ingredients** — typically a list of them. Then there is some **processing** to be done: chopping some things, grinding others. After that there is a **specific sequence** in which things go into the pot — some oil first, then something else, and so on.
Certain things can be done **in parallel**: one person can chop the vegetables while somebody else grinds something up. But other things must follow a strict sequence. So cooking recipes also impose a natural dependency on the tasks needed to prepare a dish.

Example 9.6 — project planning

A typical **construction project**, or any other large project with many phases: some phases can be done in parallel, others must be done in sequence.

Context	Vertices	Edges
Setting up an office	the tasks	task t before task u
A degree programme	the courses	prerequisite before course
A recipe	ingredients and steps	step must precede step
A construction project	the phases	phase must precede phase

Once we have modelled these things as DAGs, we can solve a similar problem arising across all these different applications by a **uniform** problem on DAGs — namely **topological sorting** and **longest paths**. That is what we shall be looking at.

Lecture 10 — Topological Sorting

Why cycles make it impossible; every DAG has an indegree-0 vertex; the algorithm; a full worked trace

1 What a topological sort is

We have motivated the use of DAGs by saying that they are useful for representing tasks and dependencies. One of the things we need to do in a directed acyclic graph is to arrange the vertices in a list that **respects the dependencies** — a **topological sort**.

Topological sort

Sequentially list out the vertices in such a way that every time there is a directed edge from i to j,

i must appear before j in the sequence.

In terms of our applications, if these are tasks and dependencies, then a topological sort represents a **feasible schedule**: a schedule in which no task appears before all the tasks it depends on are completed. When you come to do something, everything you needed to do before it is already done.

2 Why cycles make it impossible

The first thing to notice is that if there are cycles, we **cannot** do this. Informally we argued that if I depend on somebody else to finish, and they depend on me, then nothing can happen because we are waiting for each other. Let us see formally what it means.

2.1 The requirement is transitive

It is not just for every *edge* (i, j) that a topological sort requires i before j. If we have a **path** of dependencies — i depends on k and k depends on j — then i must come before k, and k must come before j, so **transitively** i must come before j.

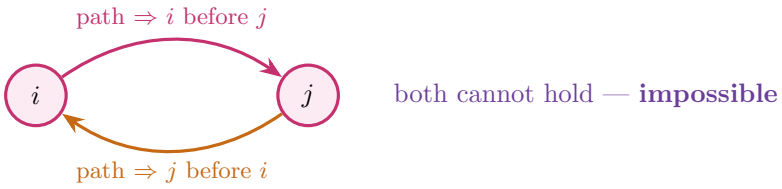
Any time there is a **path** from i to j, i must be listed before j.

2.2 The contradiction

Now suppose we have a cycle in a directed graph. Remember that a cycle cannot be something which goes from i to i without visiting any other vertex — we need to cross at least one edge. So we must go from i to some other vertex j and then come back. That means

there is a path from i to j, and a path from j back to i.

By the requirement above, the first path obliges the topological sort to put i before j; but the second path obliges it to put j before i. Clearly, in a given sequence either i comes before j or j comes before i — we cannot satisfy both constraints, so this is impossible.



So if we have a cycle of dependencies, there is no way to order them in a feasible sequence such that each task appears only after everything it depends on has happened.

3 The other side: every DAG can be sorted

We have shown that if the graph is directed *with* cycles, topological sorting is always impossible. What we now need to argue is the converse:

> If you give me a feasible set of constraints — no cyclic dependencies, so a **DAG** — then I will always be able to complete it in some reasonable order. **Every DAG can be topologically sorted.**

3.1 The strategy

Clearly we have to begin somewhere. The first thing to list is a task with **no dependencies** — one that does not require anything else to be done. There must be such a vertex, and there may be more than one.

In our example graph

Vertex 0 has no incoming edges, so it does not depend on anything; vertex 1 has no incoming edges either. So we could start with 0 or with 1.

As we complete dependencies, a later vertex that depends on a few things becomes **available**, because everything that needed to be done before it is done. So the general strategy is:

The strategy

1. Start with something that has **no dependencies**.
2. Every time a dependency is satisfied, **strike it off** the list.
3. Vertices that do have dependencies eventually find all of them already listed, and can then be listed themselves.

To apply this we must **guarantee two things**: that there will always be a starting vertex with no dependencies, and that eventually every vertex will find all its dependencies listed out — so that we finish by listing them all.

4 Every DAG has a vertex of indegree 0

Recall that in a directed graph we speak of the **indegree** and the **out-degree** rather than just the degree. In an undirected graph the degree counts the edges incident on a vertex; in a directed graph those edges either come *in* or go *out*. The **indegree** is how many edges point *into* a vertex v.

What we are looking for is a vertex with no dependencies — nothing pointing into it, so there is no edge of the form (u, v).

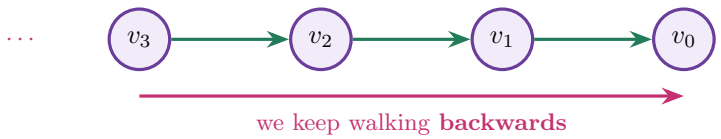
Claim

Every DAG must have a vertex with **indegree** 0.

Proof by contradiction

Suppose every vertex has indegree greater than 0. Pick any one, and call it v_0.

Since v_0 has indegree greater than 0, there is at least one edge coming into it, so we can **follow that edge backwards** to a preceding vertex v_1. But v_1 also has indegree greater than 0, so we cannot stop — we go back to another vertex v_2. And so on.



Backward walks in our graph

Starting at 6 we can walk back and stop at 0. Starting at 7 we could follow a different path — 7 back to 4 and then to 0. Or from 6 we could have gone back through 5, then 2, then 1. Whichever way we do it, all these paths eventually **stop**. Why?

Now, **if we ever hit the same vertex again, we have a cycle.** So assume v_1 is different from v_0, v_2 is different from both, and so on — we keep hitting **new** vertices as we go. (If we did not hit a new vertex on some backward step, we would already have found a cycle, and we know this graph has none.)

But the graph has only n vertices. Starting at v_0, one step back reaches v_1, two steps reach v_2, and $n-1$ steps reach v_{n-1}. If we now take an n-th step backwards, we **cannot find a new vertex**, because all n vertices have already been traversed somewhere along the path. So the next vertex must be one already seen — and therefore there is a **directed cycle**.

Since a DAG cannot have a directed cycle, this is impossible. Hence the assumption was wrong, and **there must be a vertex with indegree** 0.
(In short: assume everything has non-zero indegree; then we can find a path of arbitrary length, in particular of length n, visiting $n+1$ vertices; and $n+1$ vertices must repeat one, so there is a cycle.) ■

5 Deleting a vertex leaves a DAG

So we always have a starting point. In our graph, vertices 0 and 1 both have no edges pointing into them. What do we do? We **list it out**, and then pretend it is no longer a constraint, because it has been done.

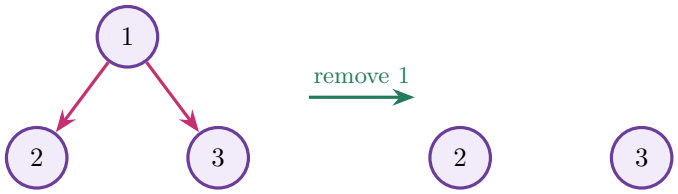
Example 10.1

Vertex 4 requires vertex 0 to be completed. But if we list out 0, saying 0 is done, then 4 no longer has that constraint — it is gone.

So suppose we delete the vertex we just listed, together with **all the edges pointing out of it**. We are left with a smaller graph: one fewer vertex, and fewer edges depending on how many were attached.

Removing a vertex from a DAG gives a DAG

In this process we have only **removed** edges from a directed graph. If the original graph had no cycles, this one cannot either, because we have not put back any edge between two vertices that were not already connected.
So: take a DAG, remove any vertex from it, and what remains is again a DAG. It may not be connected if we chose badly — but at least it cannot have any cycles.



a DAG

still a DAG —
though now **disconnected**

Therefore, by the same argument, the new DAG must *again* have a vertex of indegree 0. At every stage we have a DAG; whenever we have a DAG there is a vertex of indegree 0; so at every stage there is at least one vertex we can remove. Repeating this, after n stages we must have removed all the vertices.

Start with a DAG → remove a vertex of indegree 0 → we have another DAG → therefore another vertex of indegree 0 → remove that, and so on.

This is why the process is guaranteed to **make progress** and to **exhaust all the vertices**.

6 Implementing it: computing the indegree

The first step in implementing this as an algorithm is to **compute the indegrees**.

6.1 From an adjacency matrix

Assume the graph is presented as an adjacency matrix. The incoming edges are in the **columns**: row i holds all the outgoing edges from i, while column i holds every entry of the form (j, i) — an edge from j into i.

Walk **down column** i and add up the 1s — that gives the **indegree of** i.

Doing this one scan, we can count the incoming arrows without looking at the picture at all. For example, in our graph vertex 2 has indegree 2, because two edges come into it.

6.2 From an adjacency list

With an adjacency list we only have **outgoing** edges: the list for i holds only the edges pointing out of i. So how do we get the edges pointing *into* i? Not in one shot — instead we scan across **all** the lists.

Building the indegrees from the lists

Keep a separate counter for the indegree of every vertex, all starting at 0. Then scan the lists from top to bottom, and for each outgoing edge you see, go to the corresponding **target** and increment its counter by 1.

Start with vertex 0, whose list is $2, 3, 4$: add 1 to the indegree of 2, of 3 and of 4. Then vertex 1, whose list is $2, 7$: add another 1 to the indegree of 2, and 1 to that of 7. And so on.

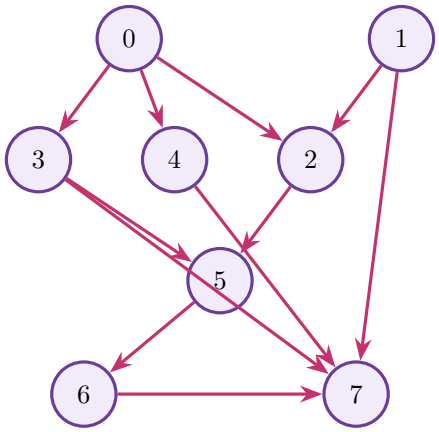
So even with an adjacency list a simple scan sets up all the indegrees. After that it is only a matter of checking the indegrees as we delete vertices, so it does not matter whether the edges are recorded as incoming or outgoing.

The usual caveat on cost

As before: with an **adjacency matrix** everything takes order n time per vertex, because we must look at a whole row or column; with an **adjacency list** we can do it in time proportional to the total number of **edges**.

7 A full worked trace

Here is the graph we shall sort, on eight vertices 0 to 7:



Scanning the columns gives the starting indegrees:

Initial indegrees

vertex	0	1	2	3	4	5	6	7
indegree	**0**	**0**	2	1	1	2	1	4

We know in advance that at least one vertex must have indegree 0; we do not know which, but we find it by **scanning down the list looking for a** 0. Here there are two, namely 0 and 1.

Updating the indegrees efficiently

When we remove a vertex i we must **simultaneously update** the indegrees — but we do *not* have to rescan them all. We look only at the **row for** i, and for every edge (i, j) we reduce the indegree of j by 1.

The trace

Removing vertices one at a time

Step	List out	Reduce indegree of	Now available (indegree 0)
1	1	2 and 7	0
2	0	2, 3 and 4	2, 3, 4 — a wide choice
3	3	5 and 7	2, 4
4	2	5	4, 5
5	5	6	4, 6
6	6	7	4 only — 7 still has indegree 1
7	4	7	7
8	7	—	**done**

Following it through: we choose 1 first, list it, and remove it; the edges out of 1 go to 2 and 7, so both of those indegrees drop. Now we have a smaller DAG, and here we have **no choice** — only 0 has indegree 0. Removing 0 reduces the indegrees of the three vertices it pointed to, and now a **wide variety** of vertices have indegree 0, since they all depended on 0 and 1.

We pick 3; it was pointing at 5 and 7, so both of those reduce. Now we have two choices, 2 and 4; we take 2, reducing the indegree of 5. Now 4 and 5 are both candidates, so we take 5, reducing the indegree of 6. Then 6, reducing the indegree of 7 — but we still cannot enumerate 7, because it has indegree 1 left, while 4 has indegree 0. So we take 4, which reduces 7 to 0, and finally we enumerate 7.

The topological sort obtained: 1, 0, 3, 2, 5, 6, 4, 7

Perhaps not the most obvious sequence one would have thought of — we might have expected to go top to bottom as $0, 1$, then $2, 3, 4$, then $5, 6, 7$. But this is a **valid** sequence.

8 More than one topological sort is possible

The other thing to notice is that a topological sort is **not unique**. We saw this when setting up the room: tiling the floor and plastering the walls can be done in either order. What happens is that we end up with **multiple choices** of indegree-0 vertices.

- In the example above we could have started with 0 or with 1; we chose 1, and starting with 0 would have given a slightly different sequence.
- Later we had 2, 3 and 4 all available with indegree 0. We chose 3 first, but could equally have done 2 first, or 4 first.

Whenever there are multiple vertices with indegree 0, the topological sort does not force us to take one or the other. We might choose to take the smallest, in which case we get one particular order — but **multiple orderings are possible**.

So a topological sort produces a sequence which is **compatible** — but it is by no means the *only* sequence.
In the extreme case where there are **no dependencies at all**, any ordering is possible. With n independent tasks there are

$$n!$$

orderings. The number of topological orderings can therefore be very large — and we are not interested at this point in *counting* them, only in **finding one**.

9 Summary

Point	What we found
DAGs and dependencies	a natural way to represent them; the fundamental problem is finding a feasible schedule that violates no constraint
Cycles	make a topological sort impossible, since a path each way would demand i before j and j before i
Indegree 0	every DAG has at least one such vertex — proved by contradiction, walking backwards until a vertex must repeat
Deleting a vertex	leaves another DAG, possibly disconnected, so the argument applies again and the process always makes progress
Implementation	indegrees from columns (matrix) or by scanning all lists (list); on removing i, decrement along the row for i
Non-uniqueness	multiple indegree-0 choices give multiple valid orderings — up to $n!$ when nothing depends on anything

References and Acknowledgement

These notes are based on the video lectures of Mathematics–1 offered by the IIT Madras BS Degree Programme.

Notes created by: Vishal (Instructor)